

LEVEL 8 PROJECT DOCUMENTATION

AWS INFRASTRUCTURE AS CODE WITH TERRAFORM AND CLOUDWATCH MONITORING

Terraform + AWS VPC + ALB + Auto Scaling + ECR + EC2 + Tomcat + CloudWatch + SNS

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Environment: Amazon Linux 2023 / EC2 / Docker / Tomcat
Infrastructure Tool: Terraform

Prepared By Manimaran

Table of Contents

- 1. Project Overview
- 2. Problem Statement
- 3. Objectives
- 4. Technologies and AWS Services
- 5. Prerequisites
- 6. System Architecture
- 7. Complete Terraform and Monitoring Workflow
- 8. Project Components
- 9. Terraform Project Structure
- 10. Key Configuration
- 11. Complete Implementation Procedure – 66 Screenshot Actions
- 12. Monitoring and Alerting Design
- 13. Application Request and Monitoring Flows
- 14. Verification Checklist
- 15. Troubleshooting / Important Observations
- 16. Security Notes
- 17. Key Lessons Learned
- 18. Interview / Viva Explanation
- 19. Conclusion
- Appendix – Complete Infrastructure Flow

1. Project Overview

This project demonstrates Infrastructure as Code (IaC) on AWS using Terraform, with a Java web application deployed in Docker on Amazon EC2 behind an Application Load Balancer (ALB) and managed through an Auto Scaling Group (ASG). The application image is stored in Amazon ECR, pulled by EC2 using an IAM instance role, and served by Tomcat on port 8080. The monitoring layer uses the Amazon CloudWatch Agent for additional EC2 metrics and log collection. CloudWatch alarms monitor EC2 CPU utilization, EC2 status checks, ALB 5XX responses, unhealthy targets and ALB response time. Amazon SNS forwards alarm notifications to an email subscription.

2. Problem Statement

Manually creating and monitoring a multi-tier AWS application environment requires repeated configuration of networking, security, compute, load balancing, container registry, monitoring and notification resources. This project addresses that operational complexity by defining the environment as Terraform code and adding CloudWatch-based monitoring, alarms, dashboards and SNS notifications.

3. Objectives

- Create AWS networking infrastructure using Terraform.
- Build a VPC with public and private subnets across two Availability Zones.
- Create Internet Gateway and NAT Gateway connectivity.
- Create separate security groups for the ALB and application EC2 instances.
- Create an Application Load Balancer and target group for Tomcat.
- Create an EC2 Launch Template and Auto Scaling Group with desired capacity 2 and maximum capacity 4.
- Create an ECR repository and push the Dockerized Java/Tomcat application image.
- Attach an IAM role and instance profile so EC2 can authenticate to ECR.
- Install/configure Docker, the Tomcat application container and CloudWatch Agent through user data.
- Collect EC2 metrics and logs in CloudWatch.
- Create CloudWatch alarms and route notifications through SNS to email.
- Create a CloudWatch dashboard for operational visibility.
- Verify the application through the ALB DNS name.
- Validate that Terraform can destroy the complete environment cleanly.

4. Technologies and AWS Services

Technology / Service	Purpose / Role
Terraform	Infrastructure as Code and resource provisioning
Amazon VPC	Network isolation and subnet architecture
Amazon EC2	Compute hosts for the application
Application Load Balancer	Public HTTP entry point and traffic distribution
Auto Scaling Group	Application capacity management
Amazon ECR	Private Docker image registry
AWS IAM	Instance role and permissions
Amazon CloudWatch	Metrics, logs, alarms and dashboard
CloudWatch Agent	Additional EC2 system metrics and log collection
CloudWatch Logs	Centralized log storage
Amazon SNS	Alarm notification delivery
Docker	Application containerization
Apache Tomcat / Java	Application runtime
GitHub	Source-control location

5. Prerequisites

- AWS account with permissions for VPC, EC2, ECR, IAM, Auto Scaling, Elastic Load Balancing, CloudWatch and SNS.
- Terraform installed and initialized on the workstation.
- AWS CLI configured for the intended AWS account and ap-south-1 region.
- Java/Tomcat application and Dockerfile available for the image build.
- Terraform project files organized by infrastructure component.
- An email address available for SNS subscription confirmation.
- Awareness that screenshots may contain account identifiers or environment-specific information and should be reviewed before public sharing.

6. System Architecture

The architecture separates the public load-balancing tier from the private application tier. Terraform provisions the VPC, public/private subnets, Internet Gateway, NAT Gateway, security groups, ALB, target group, launch template, Auto Scaling Group, ECR, IAM, CloudWatch resources and SNS. The ALB receives HTTP traffic on port 80 and forwards requests to the target group, which sends traffic to Tomcat on port 8080 running inside Docker on private EC2 instances.

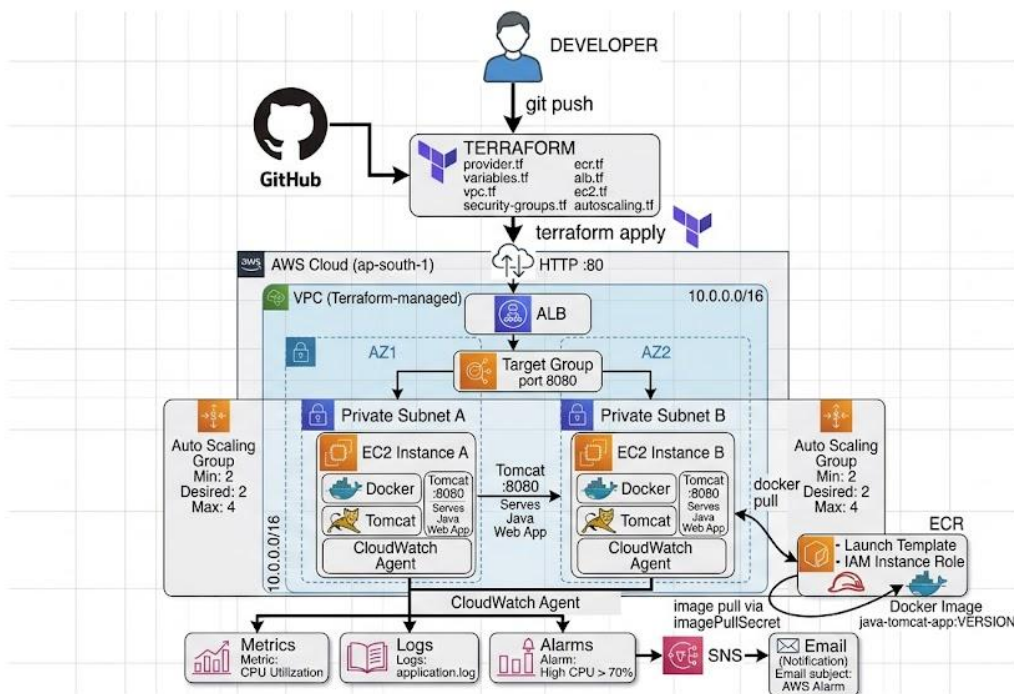


Figure 1. AWS Terraform monitoring architecture.

7. Complete Terraform and Monitoring Workflow

Step	Flow	Purpose
1	Developer → Terraform project	Infrastructure and application configuration is maintained as code.
2	Terraform → AWS	terraform apply provisions the required AWS resources.
3	VPC → Subnets → Routes	Public and private network paths are created.
4	Security Groups → ALB / EC2	Traffic is restricted according to the intended architecture.
5	ECR → EC2 IAM Role	EC2 receives permission to authenticate and pull the application image.
6	Launch Template → ASG	Instances are launched with Docker, Tomcat and monitoring bootstrap.
7	ALB → Target Group → EC2	HTTP requests are routed to healthy private application instances.
8	CloudWatch Agent → Metrics / Logs	Additional system metrics and logs are collected.
9	CloudWatch Alarms → SNS	Operational conditions trigger notification state changes.
10	SNS → Email	Alarm notifications are delivered to the confirmed email subscription.
11	CloudWatch Dashboard	Operational metrics are consolidated into a single view.
12	terraform destroy	The test environment is removed after validation.

8. Project Components

File	Purpose
provider.tf	AWS/Terraform provider configuration.
variables.tf	Reusable input variable definitions.
terraform.tfvars	Project-specific region, CIDR, instance and monitoring values.
vpc.tf	VPC, subnets, route tables, Internet Gateway and NAT Gateway.
security_groups.tf	ALB and EC2 security groups and traffic rules.
alb.tf	Application Load Balancer, target group and listener.
launch_template.tf	EC2 AMI, instance type, monitoring, IAM profile, storage and user data.
asg.tf	Auto Scaling Group configuration.
ecr.tf	ECR repository.
iam.tf	EC2 IAM role, ECR permissions and instance profile.
cloudwatch.tf	CloudWatch log group and monitoring configuration.
alarms.tf	CloudWatch metric alarms.
dashboard.tf	CloudWatch dashboard widgets.
sns.tf	SNS topic and notification configuration.
user-data.sh	Docker, CloudWatch Agent, ECR login and application startup.
outputs.tf	Important deployed resource identifiers.

9. Terraform Project Structure

The workspace separates infrastructure into component-specific Terraform files. This separation keeps networking, security, compute, registry, monitoring and notification definitions easier to review and maintain.

10. Key Configuration

Parameter	Value
AWS Region	ap-south-1
Project Name	level8-tf-aws-moni
Environment	dev
VPC CIDR	10.0.0.0/16
Public Subnet A	10.0.1.0/24
Public Subnet B	10.0.2.0/24
Private Subnet A	10.0.11.0/24
Private Subnet B	10.0.12.0/24

Parameter	Value
EC2 Instance Type	c7i-flex.large
Docker Image Tag	latest
Application Port	8080
ALB Listener	HTTP :80
ASG Minimum	2
ASG Desired	2
ASG Maximum	4
ECR Repository	level8-tf-aws-moni-webapp
CloudWatch Log Group	/aws/ec2/level8-tf-aws-moni
CPU Alarm Threshold	70%
CPU Alarm Evaluation	2 of 2 datapoints
CloudWatch Period	60 seconds
SNS Topic	level8-tf-aws-moni-alerts
Dashboard	level8-tf-aws-moni-dashboard

11. Implementation Procedure

The following 66 implementation steps are presented in chronological order. Each screenshot has an implementation action and expected result immediately before the corresponding image.

Step 1 – Terraform provider configuration

Instruction / Action: Configure the Terraform AWS provider for the project and set the AWS region to ap-south-1. Keep the provider configuration in provider.tf so the infrastructure is managed consistently.

Expected Result: Terraform has a valid provider configuration and can communicate with AWS in the project region.

```

1 resource "aws_vpc" "main" {
2   cidr_block     = var.vpc_cidr
3   enable_dns_support = true
4   enable_dns_hostnames = true
5
6   tags = {
7     Name        = "${var.project_name}-vpc"
8     Environment = var.environment
9   }
10
11 data "aws_availability_zones" "available" {
12   state = "available"
13 }
14 resource "aws_subnet" "public_a" {
15   vpc_id            = aws_vpc.main.id
16   cidr_block        = var.public_subnet_a_cidr
17   availability_zone  = data.aws_availability_zones.available.names[0]
18   map_public_ip_on_launch = true
19
20   tags = {
21     Name        = "${var.project_name}-public-subnet-a"
22     Environment = var.environment
23   }
24 }
25 resource "aws_subnet" "public_b" {
26   vpc_id            = aws_vpc.main.id
27   cidr_block        = var.public_subnet_b_cidr
28   availability_zone  = data.aws_availability_zones.available.names[1]
29   map_public_ip_on_launch = true
30
31   tags = {
32     Name        = "${var.project_name}-public-subnet-b"
33     Environment = var.environment
34   }
35 }
36 resource "aws_subnet" "private_a" {

```

Figure 2. Terraform provider configuration.

Step 2 – VPC subnet configuration

Instruction / Action: Define the VPC subnet resources in Terraform, including the two public subnets and two private application subnets used by the Level 8 architecture.

Expected Result: The subnet resources are represented in Terraform with the intended public/private layout across the configured Availability Zones.

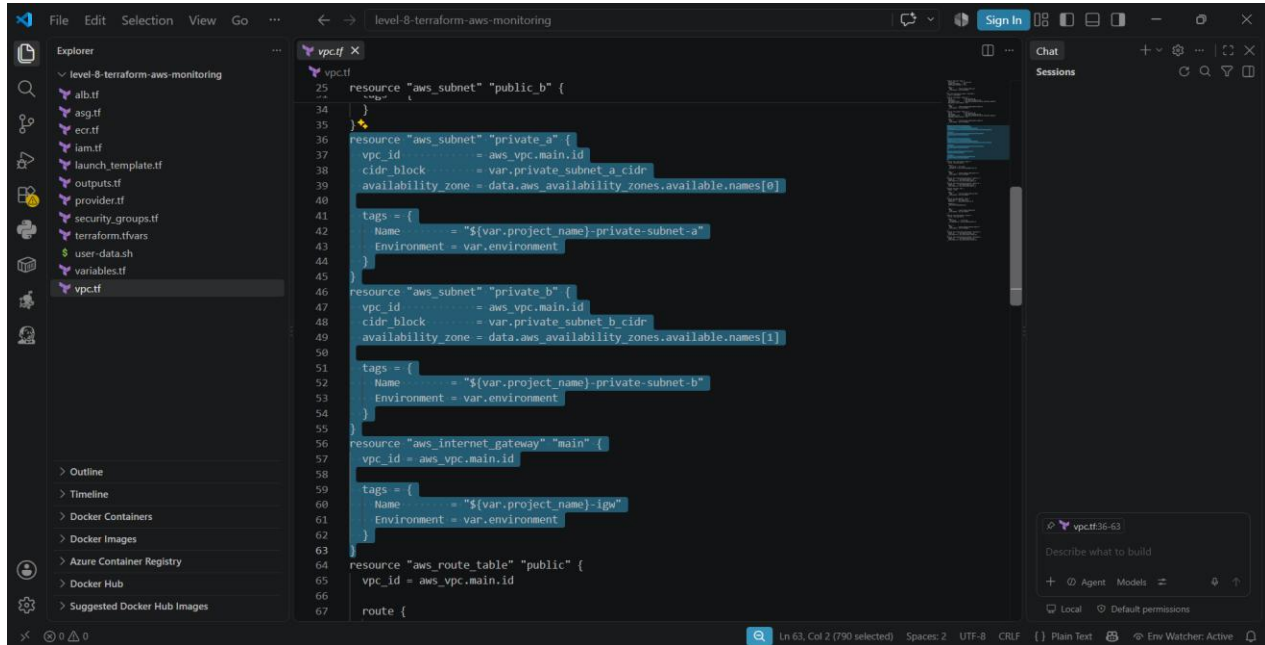


Figure 3. VPC subnet configuration.

Step 3 – Public route table configuration

Instruction / Action: Create the public route table and associate it with the public subnets. Configure the route so public resources can use the Internet Gateway.

Expected Result: The public route table and its subnet associations are present and provide the intended public network path.

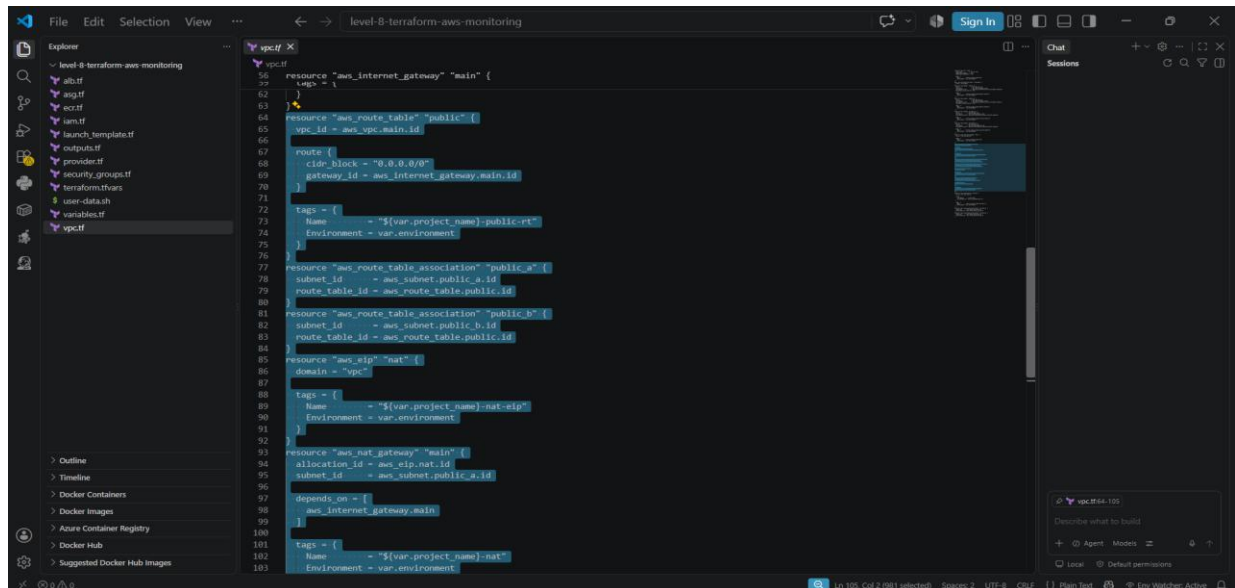


Figure 4. Public route table configuration.

Step 4 – Internet Gateway configuration

Instruction / Action: Create the Internet Gateway resource in Terraform and attach it to the project VPC. Use it as the Internet path for the public tier.

Expected Result: The Internet Gateway is associated with the VPC and is available for public-subnet routing.

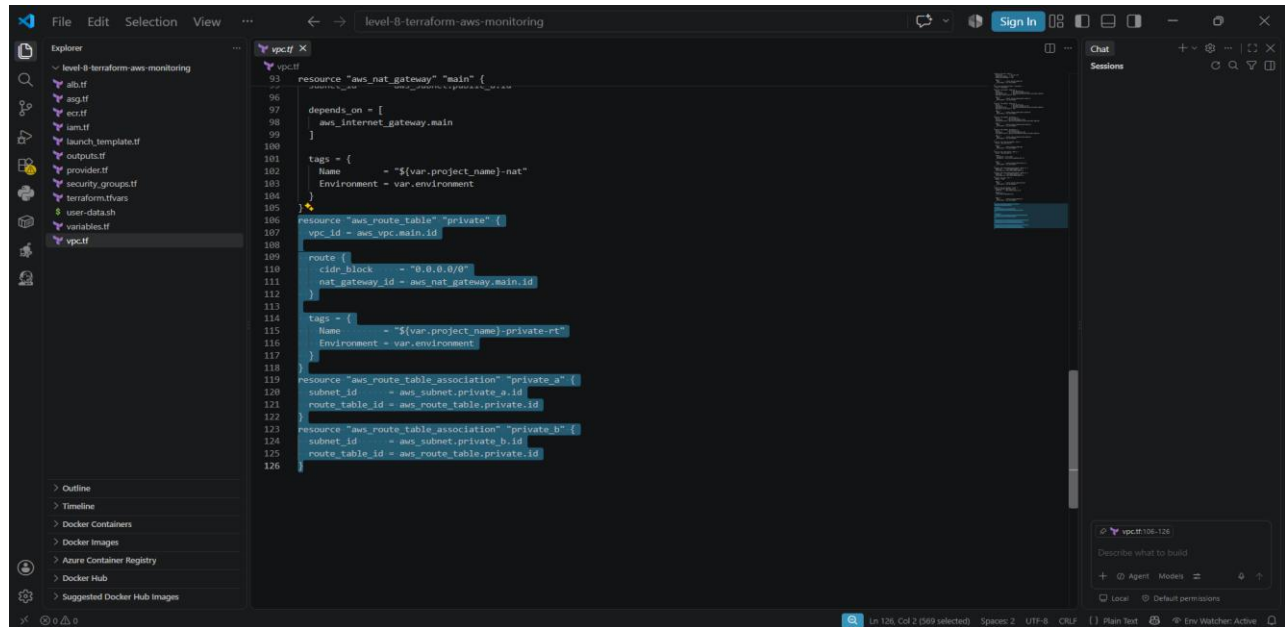


Figure 5. Internet Gateway configuration.

Step 5 – NAT Gateway configuration

Instruction / Action: Create the NAT Gateway and the required public Elastic IP association so private-subnet resources can obtain outbound connectivity without being directly Internet-facing.

Expected Result: The NAT Gateway is available and connected to the public network path for private-tier outbound access.

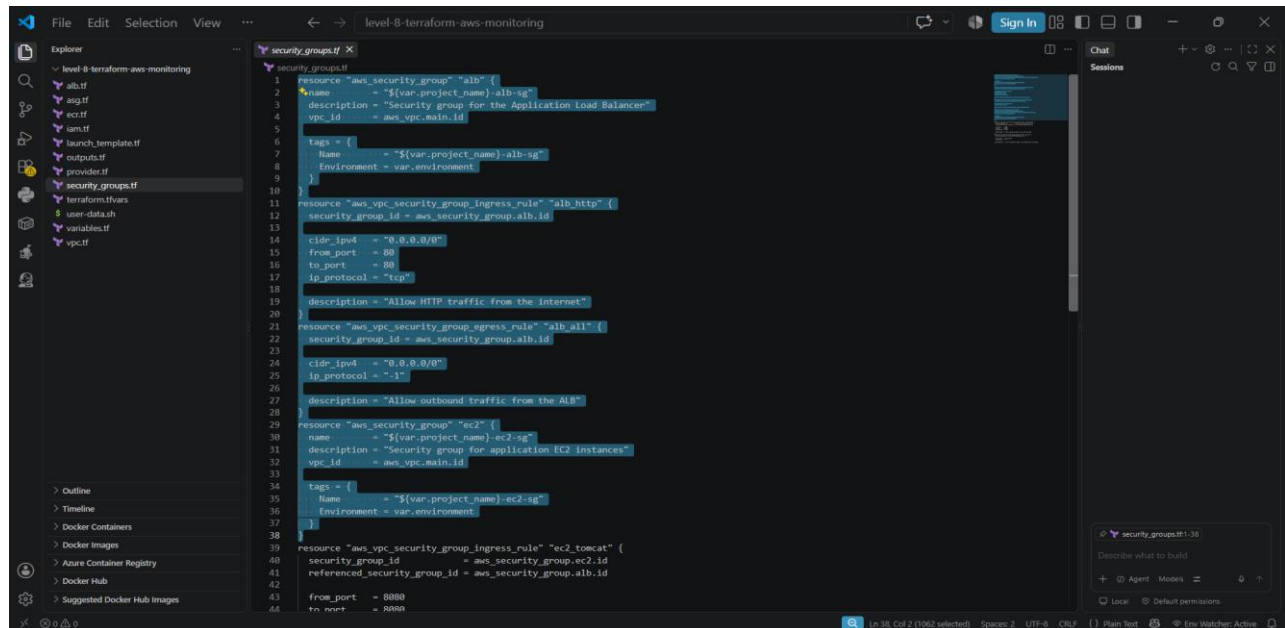


Figure 6. NAT Gateway configuration.

Step 6 – Security-group ingress configuration

Instruction / Action: Define the ALB security-group ingress rules in Terraform. Allow only the required inbound application entry traffic, with the public load balancer receiving HTTP traffic on port 80.

Expected Result: The ALB security group contains the intended ingress rule set for the load-balancing tier.

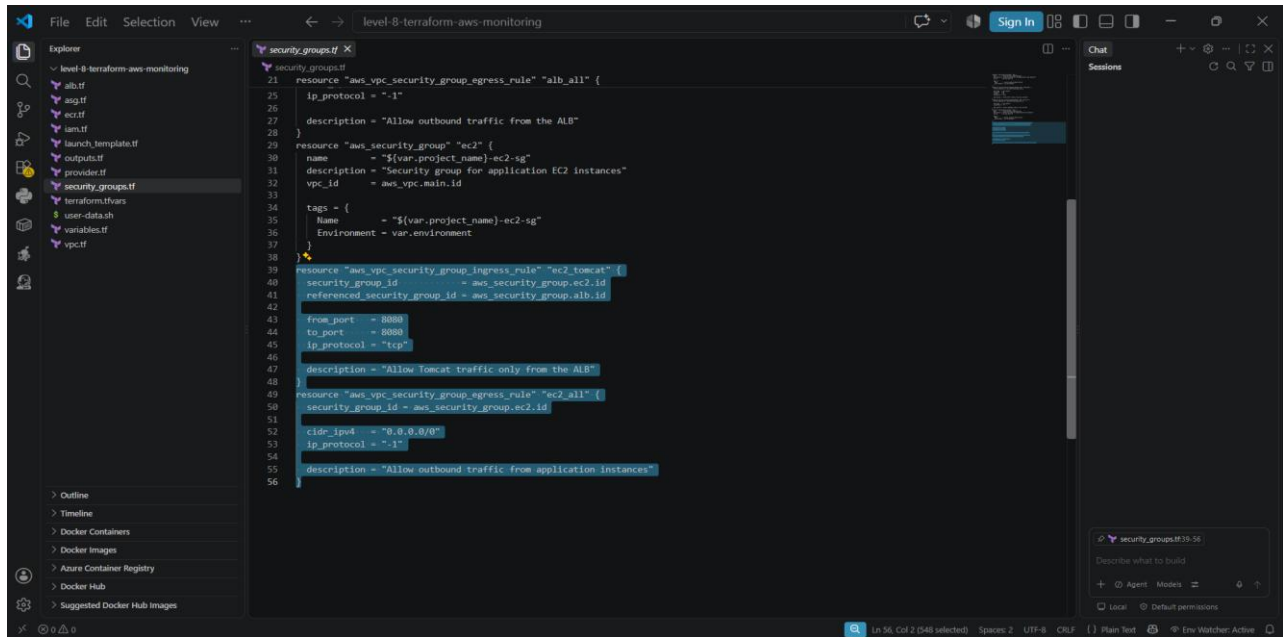


Figure 7. Security-group ingress configuration.

Step 7 – Security-group egress and EC2 security-group definition

Instruction / Action: Define the EC2/application security group and its egress rules. Restrict Tomcat traffic so the application tier accepts port 8080 traffic from the ALB security group rather than exposing it directly.

Expected Result: The application security group and its traffic relationships are defined for the ALB-to-EC2 flow.

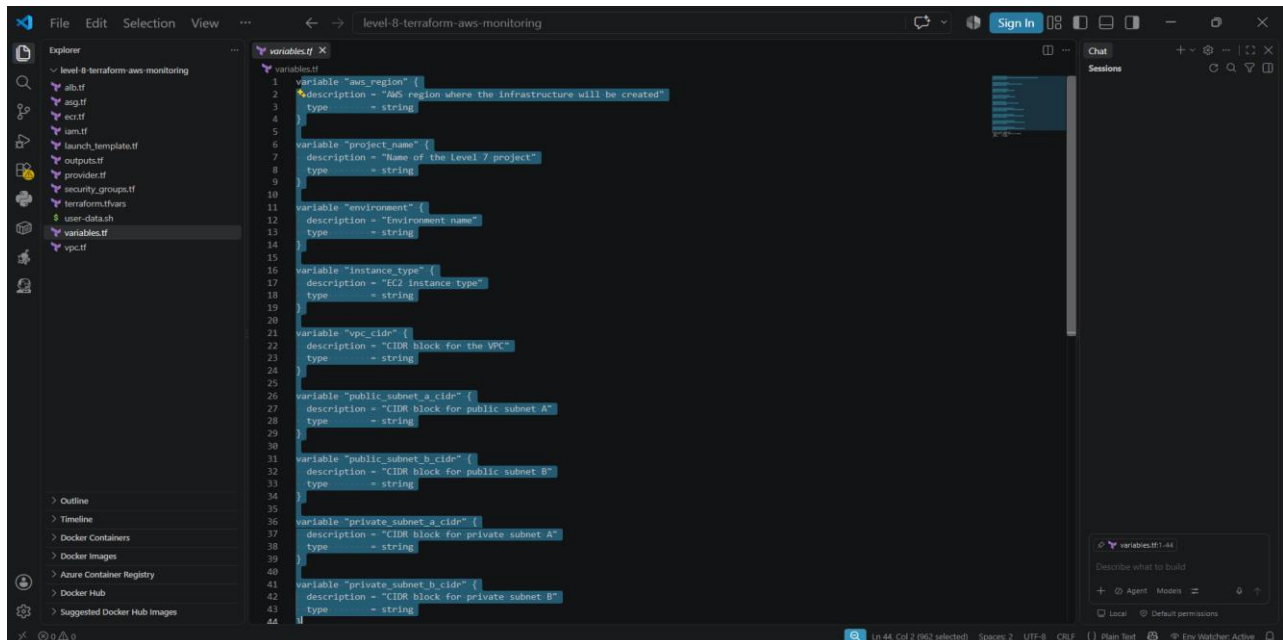


Figure 8. Security-group egress and EC2 security-group definition.

Step 8 – Terraform variables definition

Instruction / Action: Create reusable Terraform variables for project name, environment, network ranges, instance settings and monitoring values in variables.tf.

Expected Result: The infrastructure inputs are parameterized instead of being hard-coded throughout the Terraform resources.

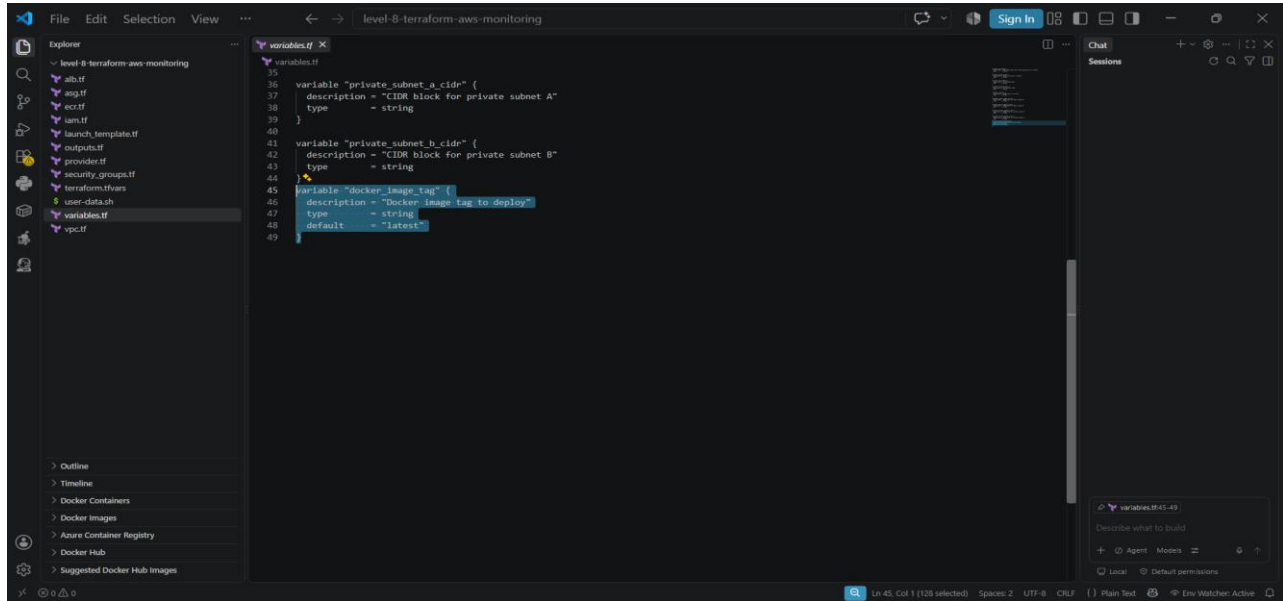


Figure 9. Terraform variables definition.

Step 9 – Private subnet variables definition

Instruction / Action: Define the private subnet variables used for the application tier, keeping the private network ranges separate from the public subnet inputs.

Expected Result: The private subnet CIDRs are available as reusable Terraform inputs.

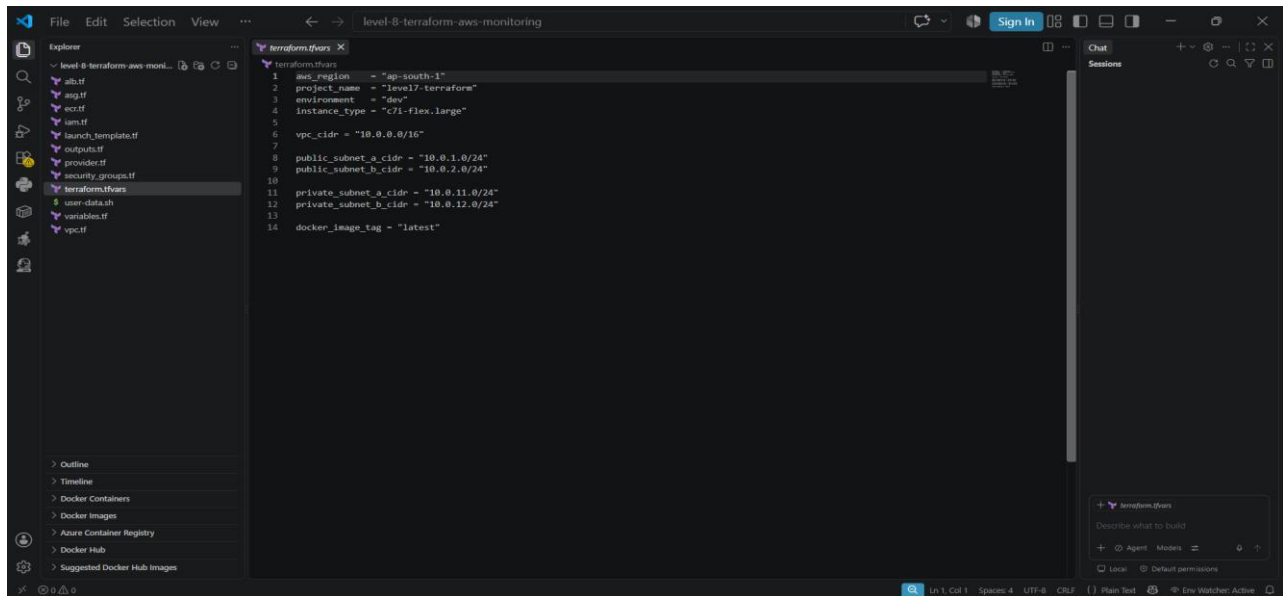


Figure 10. Private subnet variables definition.

Step 10 – Project terraform.tfvars values

Instruction / Action: Populate terraform.tfvars with the project-specific values used by this implementation, including ap-south-1, the 10.0.0.0/16 VPC and the configured subnet ranges and application settings.

Expected Result: Terraform receives the intended project values from terraform.tfvars.

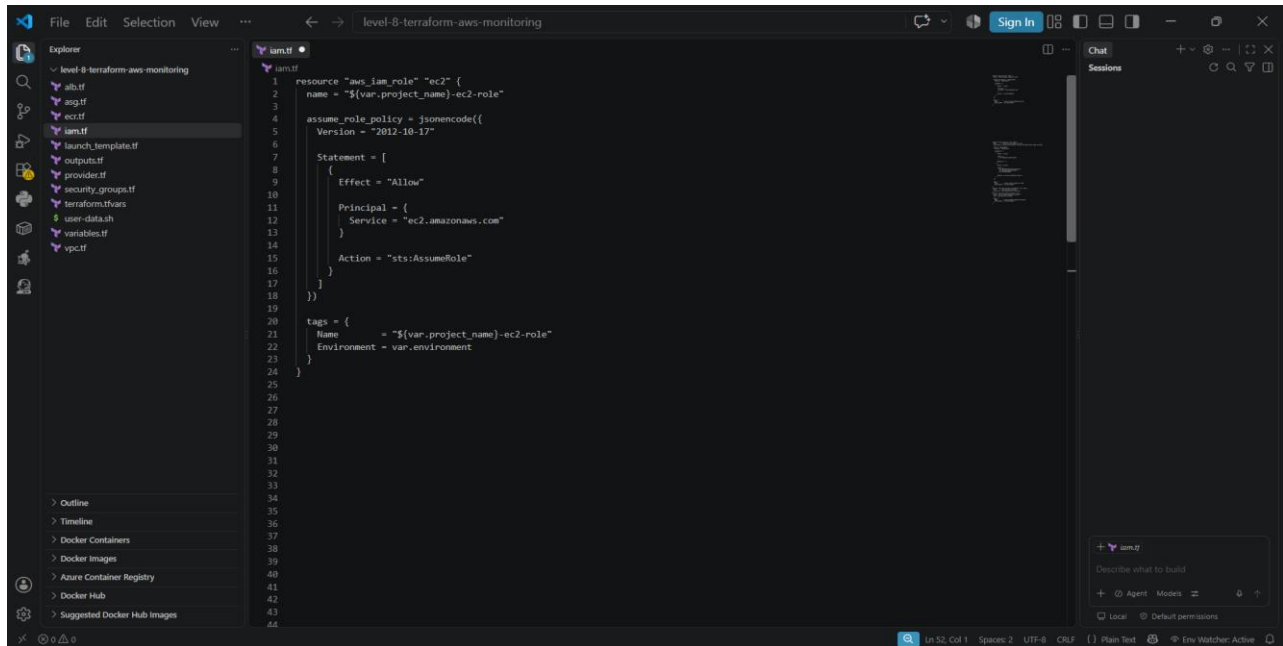


Figure 11. Project terraform.tfvars values.

Step 11 – EC2 IAM role configuration

Instruction / Action: Create the EC2 IAM role that will be assumed by the application instances. Prepare the role for EC2-side access to the AWS services required during instance bootstrap.

Expected Result: The EC2 IAM role exists and can be attached to the application instances through an instance profile.

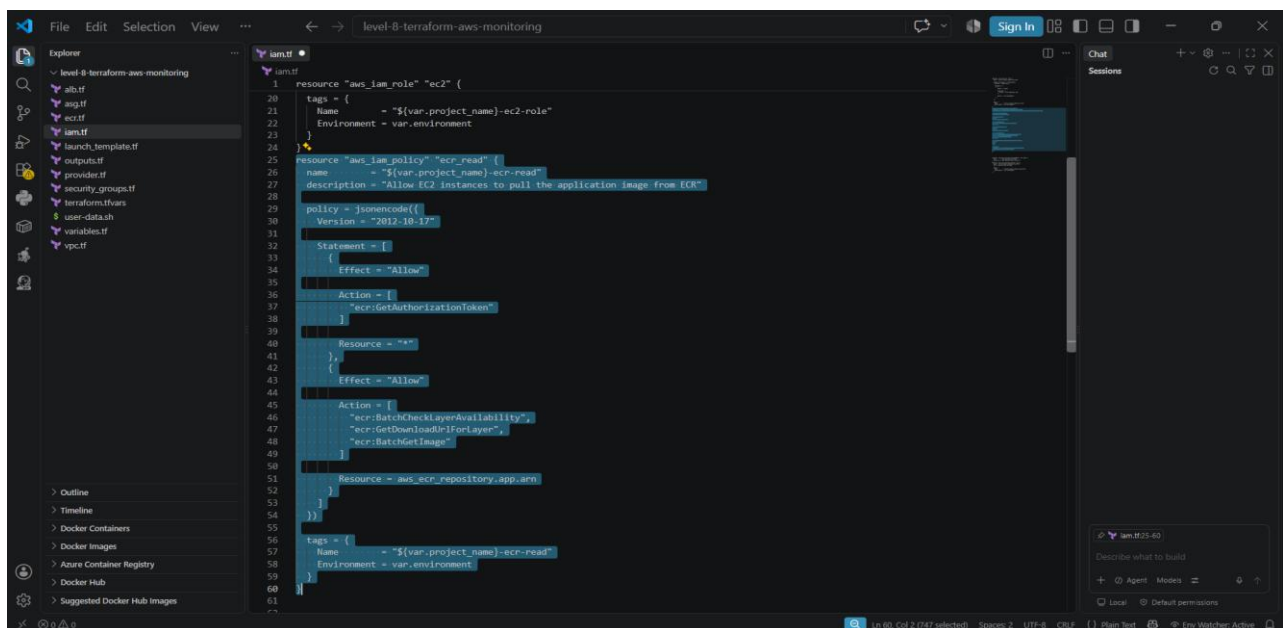


Figure 12. EC2 IAM role configuration.

Step 12 – IAM policy / permissions configuration

Instruction / Action: Define the IAM policy resources and permissions required by the application infrastructure. Keep permissions focused on the operations the EC2 instances and Terraform resources actually need.

Expected Result: The IAM policy configuration is available for attachment to the appropriate role.

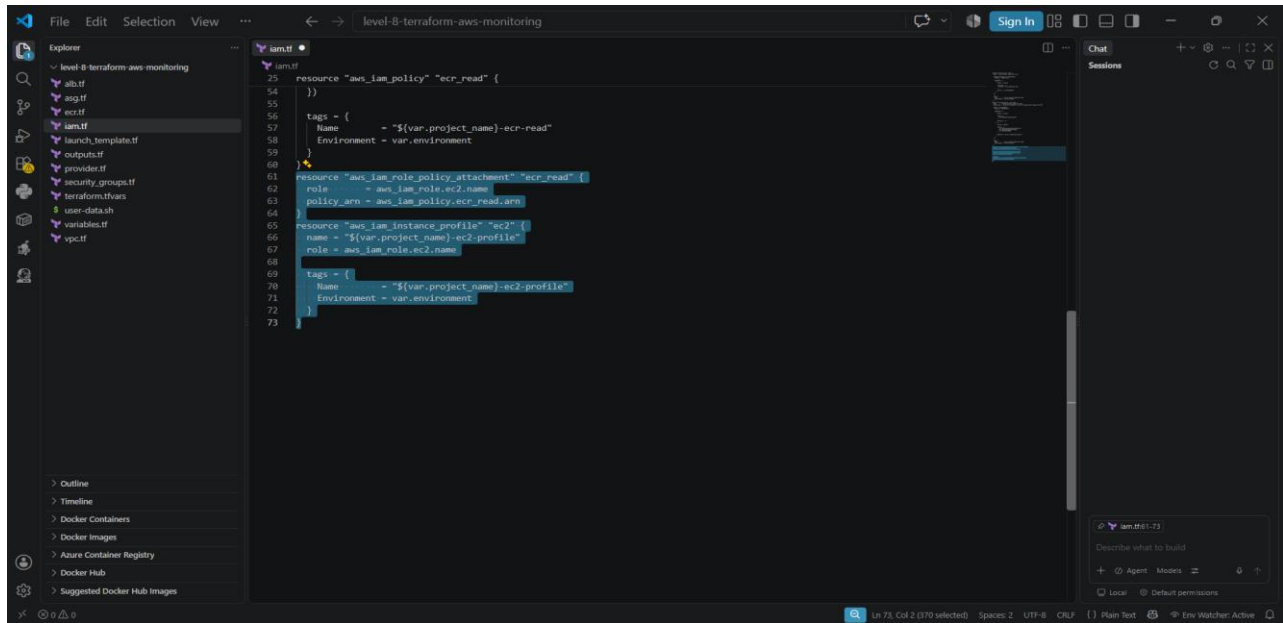


Figure 13. IAM policy / permissions configuration.

Step 13 – ECR-related IAM permission configuration

Instruction / Action: Add the ECR permissions required for EC2 to authenticate to and pull the application image from Amazon ECR.

Expected Result: The instance role contains the ECR permissions required for image retrieval.

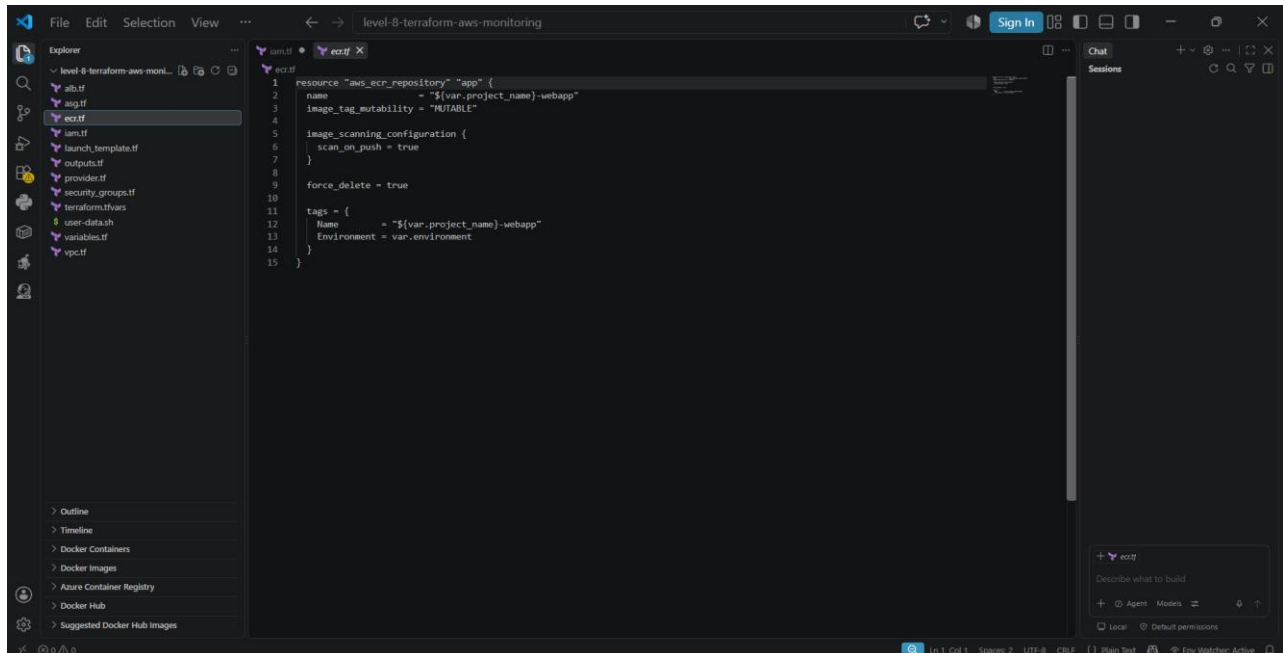


Figure 14. ECR-related IAM permission configuration.

Step 14 – ECR resource configuration

Instruction / Action: Create the Amazon ECR repository resource in Terraform. Use the project repository name level8-tf-aws-moni-webapp.

Expected Result: Terraform manages the private ECR repository used by the application.

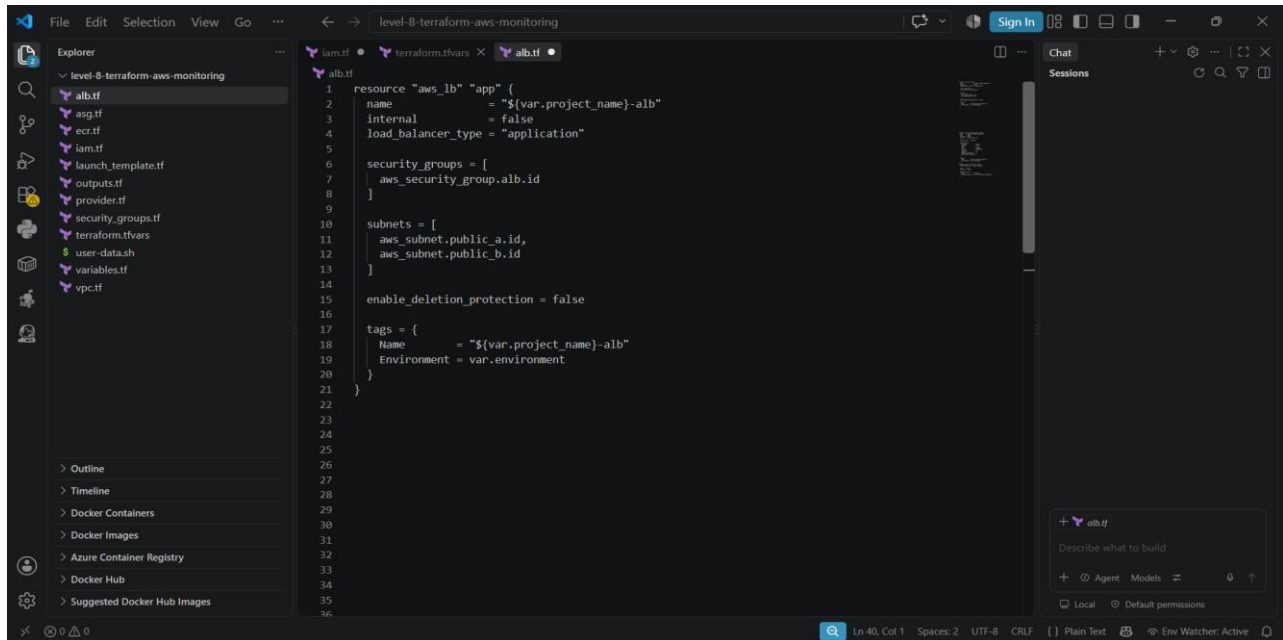


Figure 15. ECR resource configuration.

Step 15 – Application/launch infrastructure configuration

Instruction / Action: Configure the application-facing infrastructure resources that connect the ALB, target group and EC2 launch configuration.

Expected Result: The application infrastructure resources contain the references required to connect the load-balancing and compute tiers.

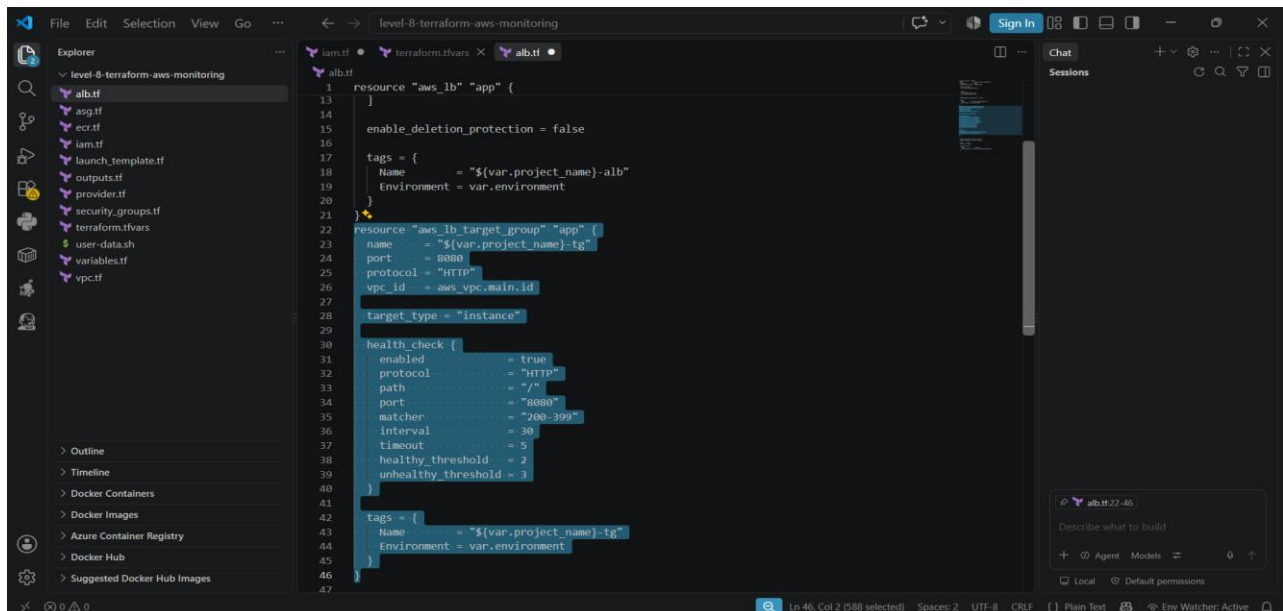


Figure 16. Application/launch infrastructure configuration.

Step 16 – Additional IAM/launch configuration

Instruction / Action: Complete the IAM and launch-related references so the EC2 launch configuration can use the required instance profile and application settings.

Expected Result: The launch configuration has the required IAM and application dependencies.

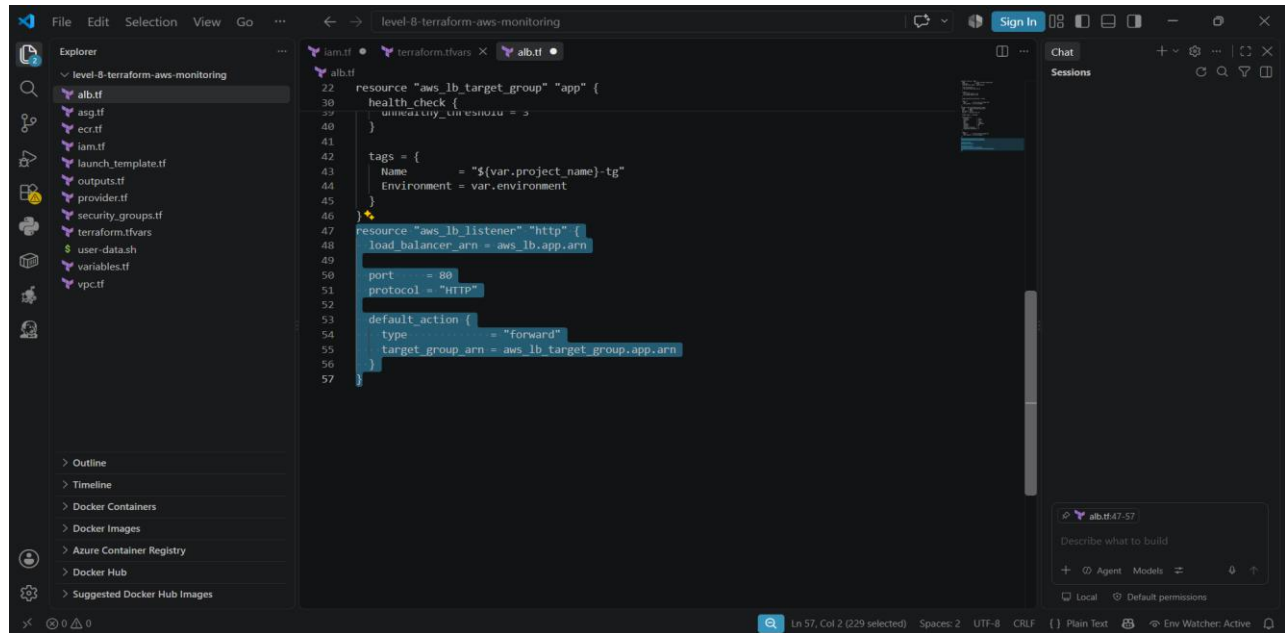


Figure 17. Additional IAM/launch configuration.

Step 17 – Launch template configuration

Instruction / Action: Create the EC2 launch template. Configure the Amazon Linux 2023 AMI, the c7i-flex.large instance type, security group, IAM instance profile, monitoring settings and startup configuration.

Expected Result: The launch template contains the required compute and bootstrap configuration for application instances.

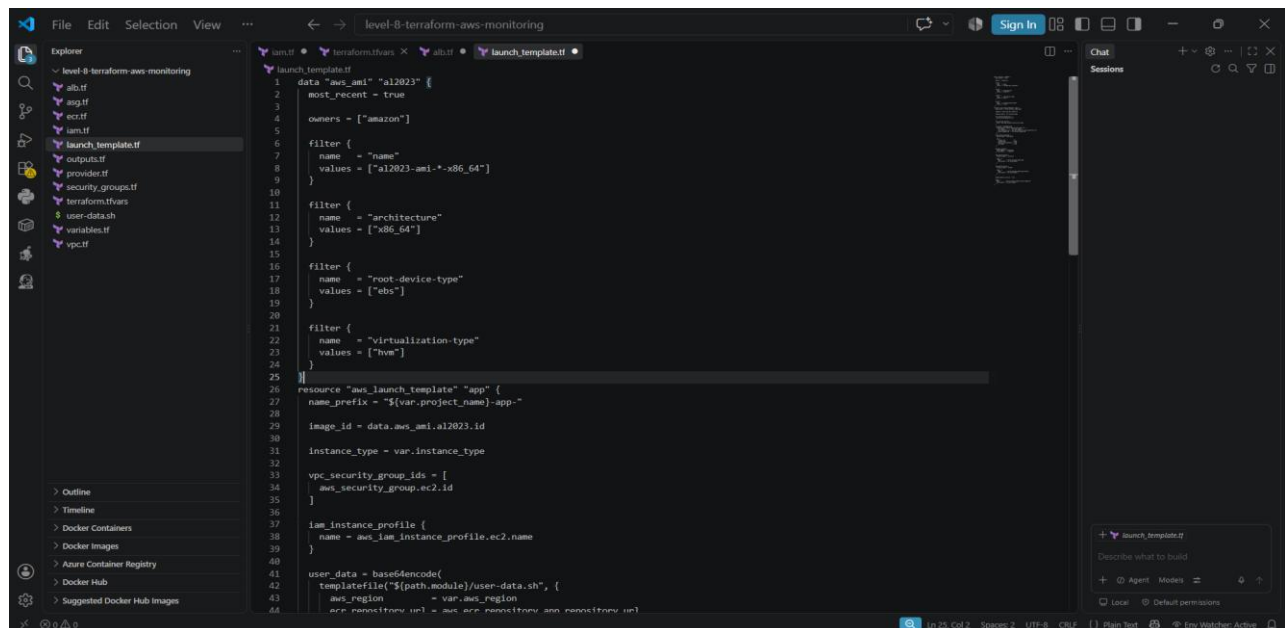


Figure 18. Launch template configuration.

Step 18 – Auto Scaling/application infrastructure configuration

Instruction / Action: Configure the Auto Scaling Group to use the launch template and target group. Set minimum capacity to 2, desired capacity to 2 and maximum capacity to 4.

Expected Result: The ASG is associated with the application target group and maintains the configured application capacity.

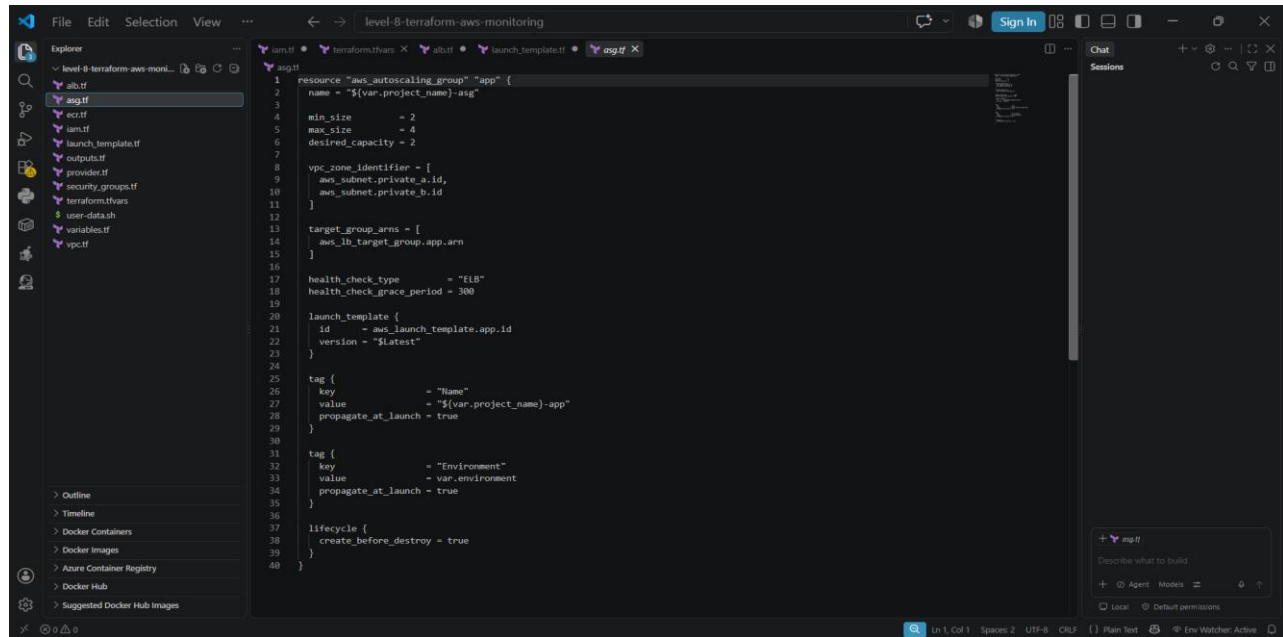


Figure 19. Auto Scaling/application infrastructure configuration.

Step 19 – CloudWatch Agent configuration in user data

Instruction / Action: Add CloudWatch Agent installation and bootstrap commands to the EC2 user-data script so new instances automatically prepare monitoring during startup.

Expected Result: A newly launched EC2 instance can install and start the CloudWatch Agent through user data.

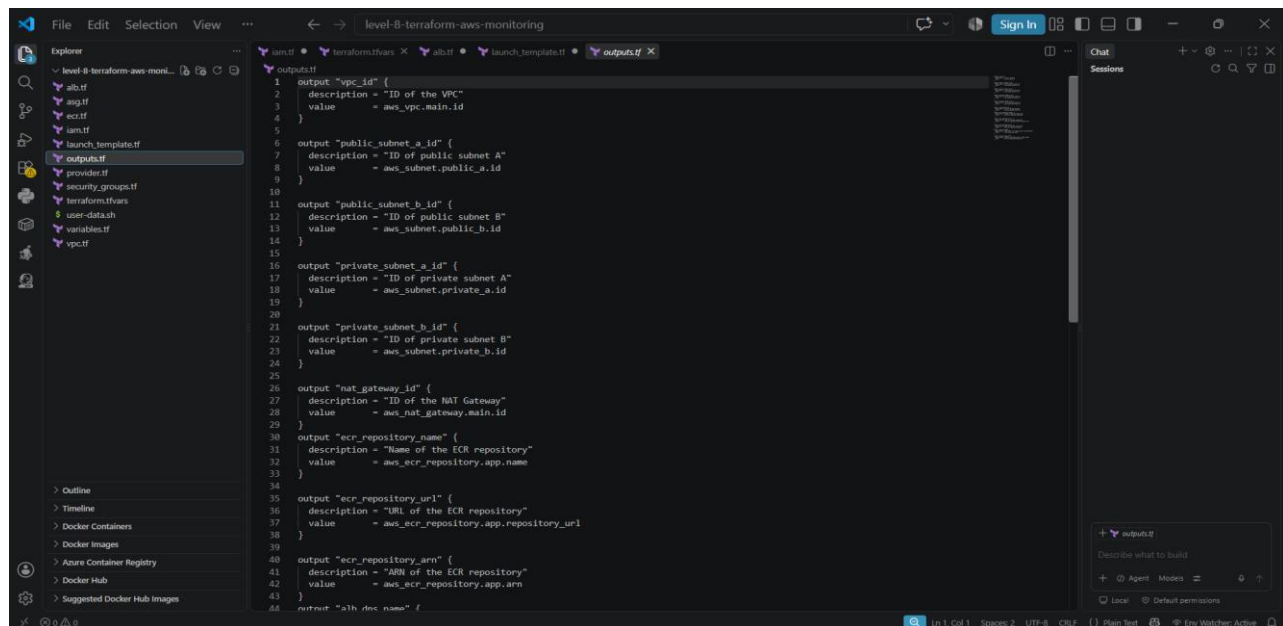


Figure 20. CloudWatch Agent configuration in user data.

Step 20 – CloudWatch Agent metrics and logs configuration

Instruction / Action: Configure the CloudWatch Agent collection settings for additional system metrics and logs, including 60-second collection and the specified memory/disk metrics and cloud-init output log.

Expected Result: The CloudWatch Agent configuration contains the intended metric and log collection definitions.

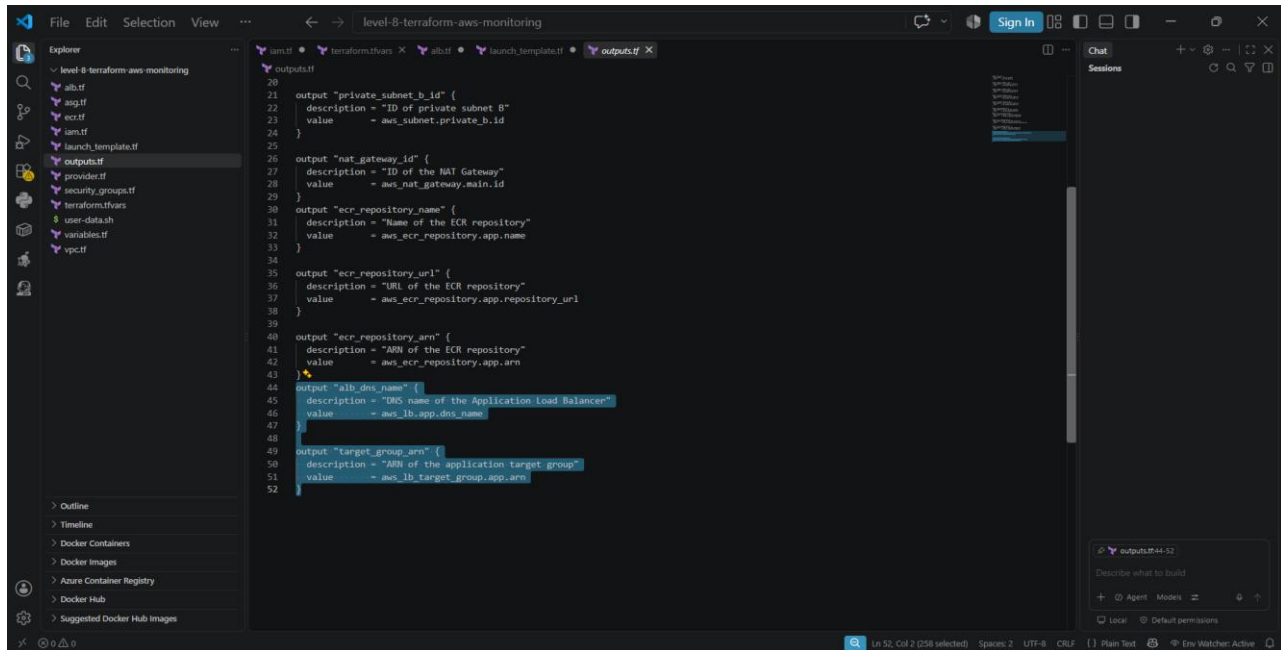


Figure 21. Evidence 20 – CloudWatch Agent metrics and logs configuration.

Step 21 – CloudWatch log group configuration

Instruction / Action: Create the CloudWatch log group used by the application environment. Use the project log group /aws/ec2/level8-tf-aws-moni.

Expected Result: The CloudWatch log group is managed by Terraform and is ready to receive instance log data.

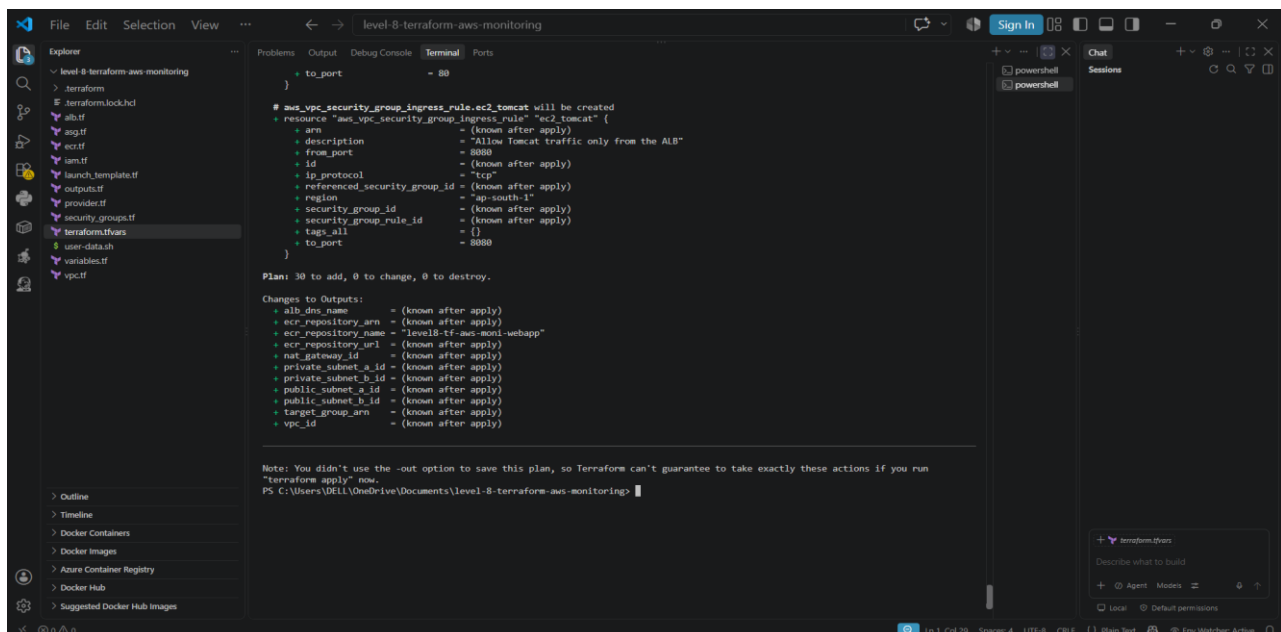


Figure 22. CloudWatch log group configuration.

Step 22 – SNS topic configuration

Instruction / Action: Create the SNS topic for monitoring notifications. Use the project topic name level8-tf-aws-moni-alerts.

Expected Result: The SNS topic is available as the notification destination for CloudWatch alarms.

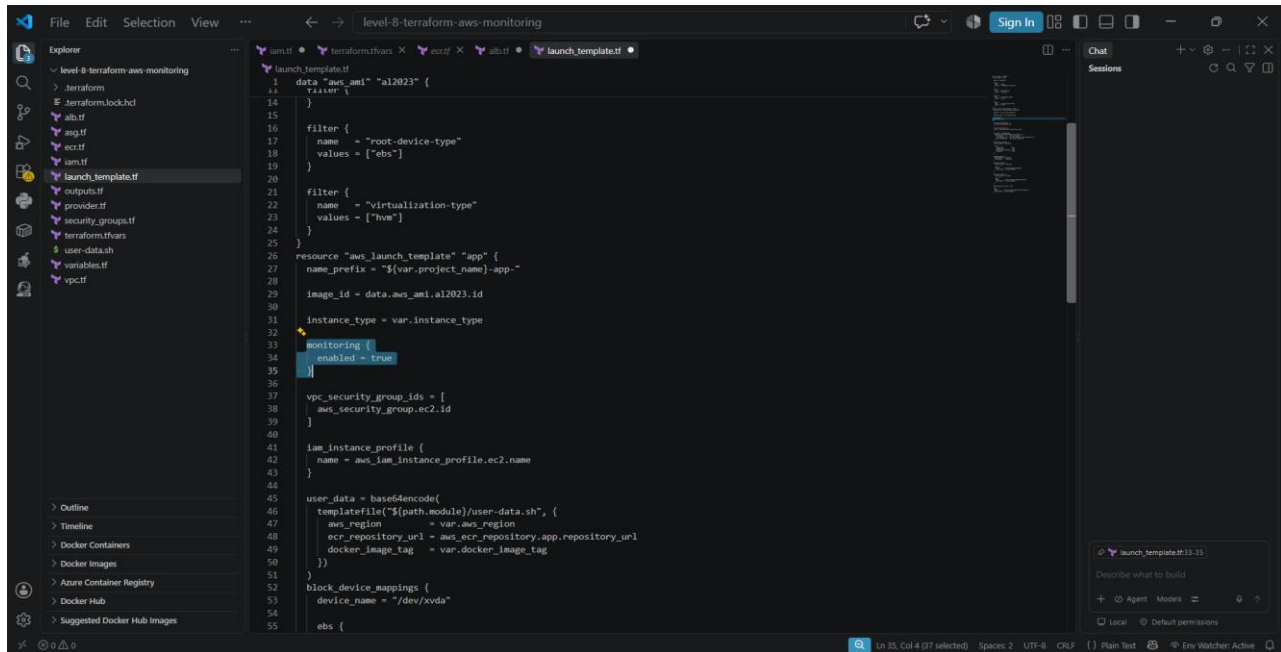


Figure 23. SNS topic configuration.

Step 23 – SNS notification configuration

Instruction / Action: Configure the SNS email subscription and notification relationship so CloudWatch alarm state changes can be delivered to the configured email address.

Expected Result: The SNS notification configuration is ready for subscription confirmation and alarm delivery.

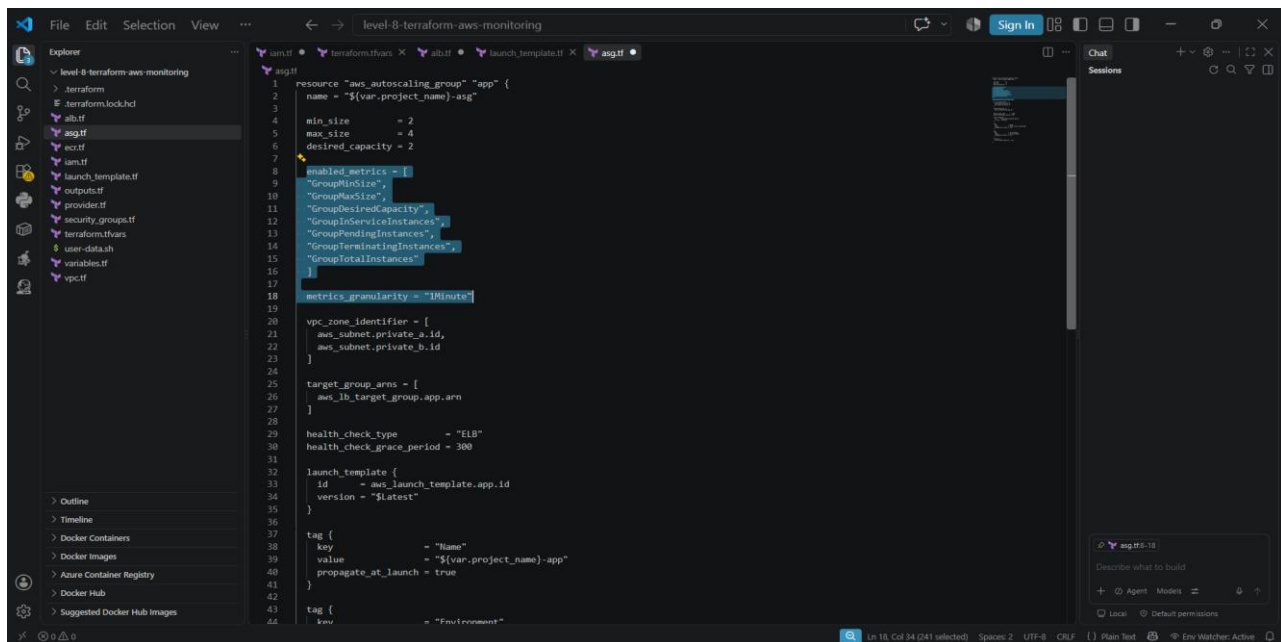


Figure 24. SNS notification configuration.

Step 24 – Terraform variables including Docker image tag

Instruction / Action: Set the Docker image tag as a Terraform variable and pass the selected image tag into the EC2 bootstrap and ECR pull configuration.

Expected Result: The application image version/tag can be changed through Terraform input rather than editing multiple resources.

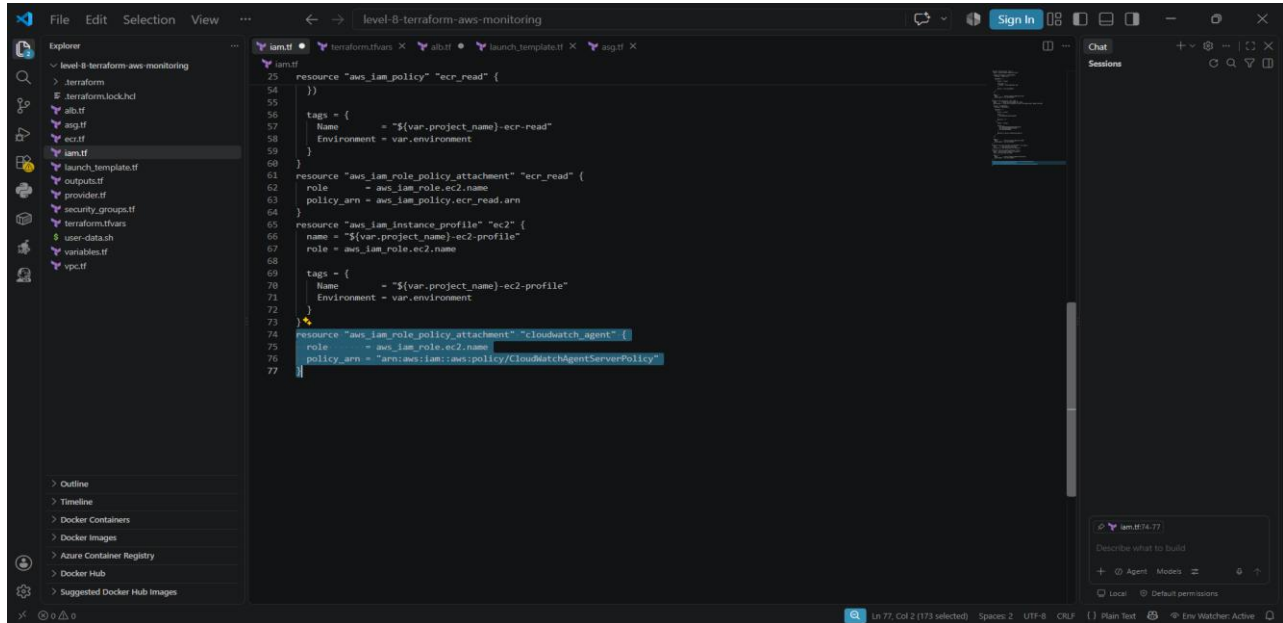


Figure 25. Terraform variables including Docker image tag.

Step 25 – Launch template monitoring and IAM profile configuration

Instruction / Action: Complete the launch-template monitoring and IAM profile settings so instances start with EC2 monitoring enabled and the correct instance profile.

Expected Result: The launch template contains the required monitoring and IAM configuration.

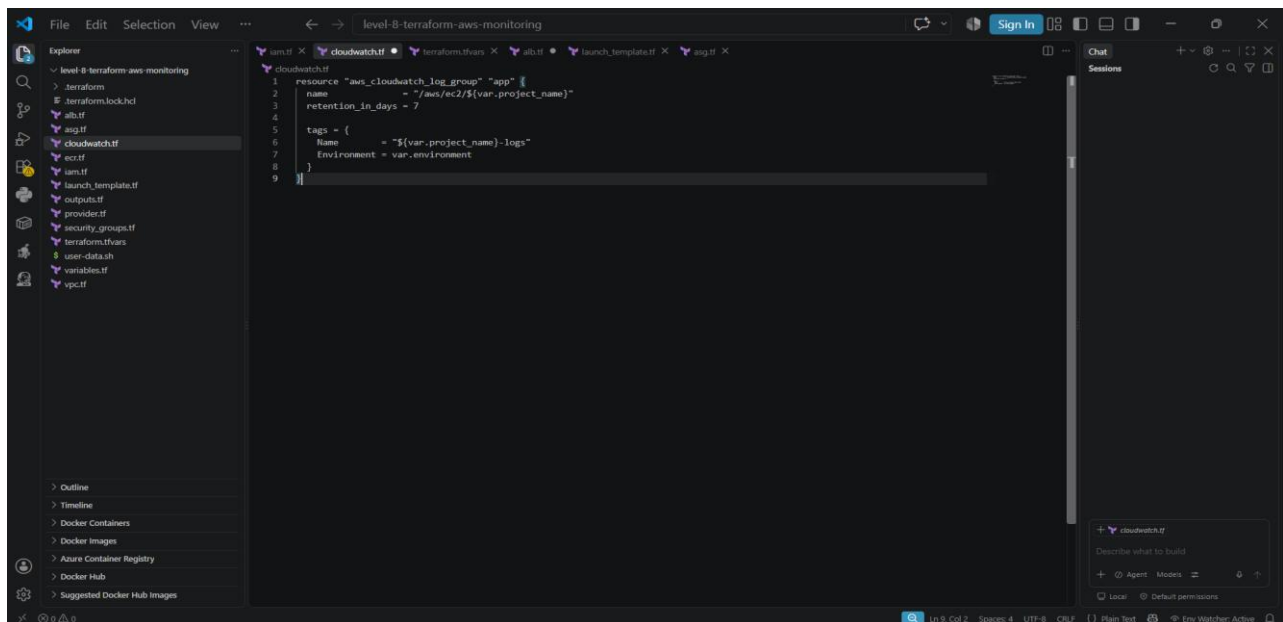


Figure 26. Launch template monitoring and IAM profile configuration.

Step 26 – SNS topic definition and related Terraform configuration

Instruction / Action: Define the SNS topic in Terraform and connect it to the monitoring resources that need to publish alarm state changes.

Expected Result: Terraform can reference the SNS topic ARN from CloudWatch alarm actions.

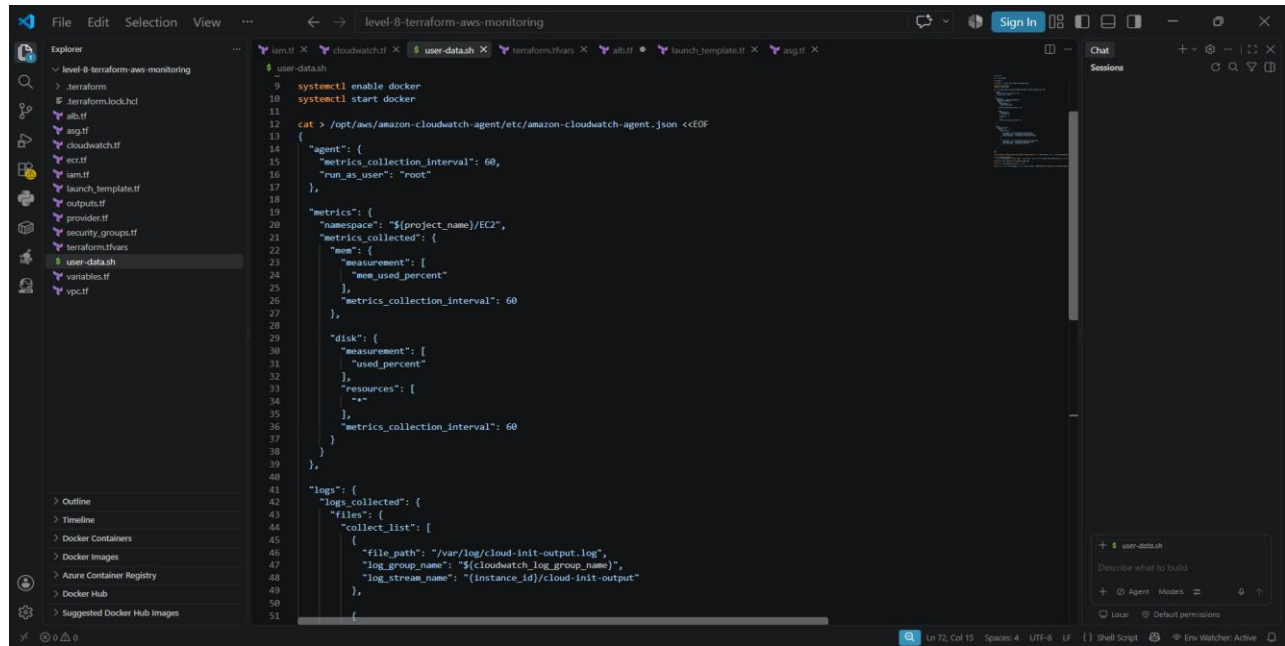


Figure 27. SNS topic definition and related Terraform configuration.

Step 27 – CloudWatch CPU alarm configuration

Instruction / Action: Create the EC2 CPU utilization CloudWatch alarm. Configure the documented 70% threshold, 60-second period, two evaluation periods and two datapoints to alarm, with SNS actions.

Expected Result: The CPU alarm is defined to notify when the configured high-CPU condition is observed.

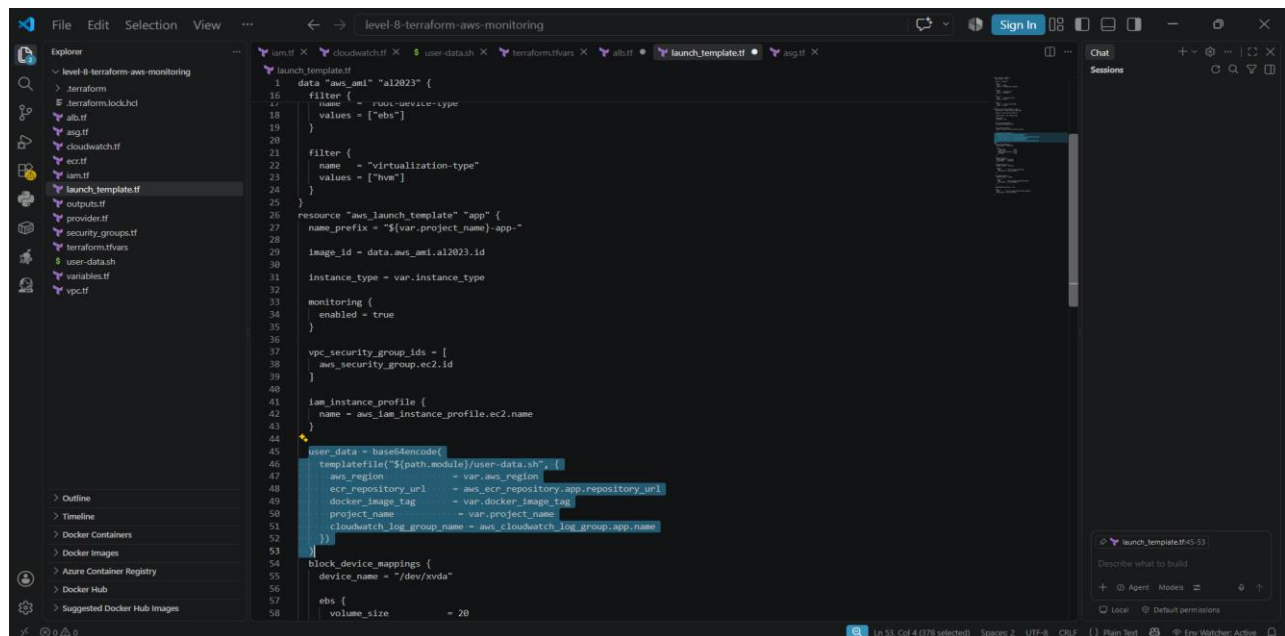


Figure 28. CloudWatch CPU alarm configuration.

Step 28 – Additional CloudWatch alarm rules

Instruction / Action: Add the additional CloudWatch alarm rules, including the EC2 status-check monitoring described in the project.

Expected Result: The additional infrastructure-health alarm definitions are present and connected to the notification path.

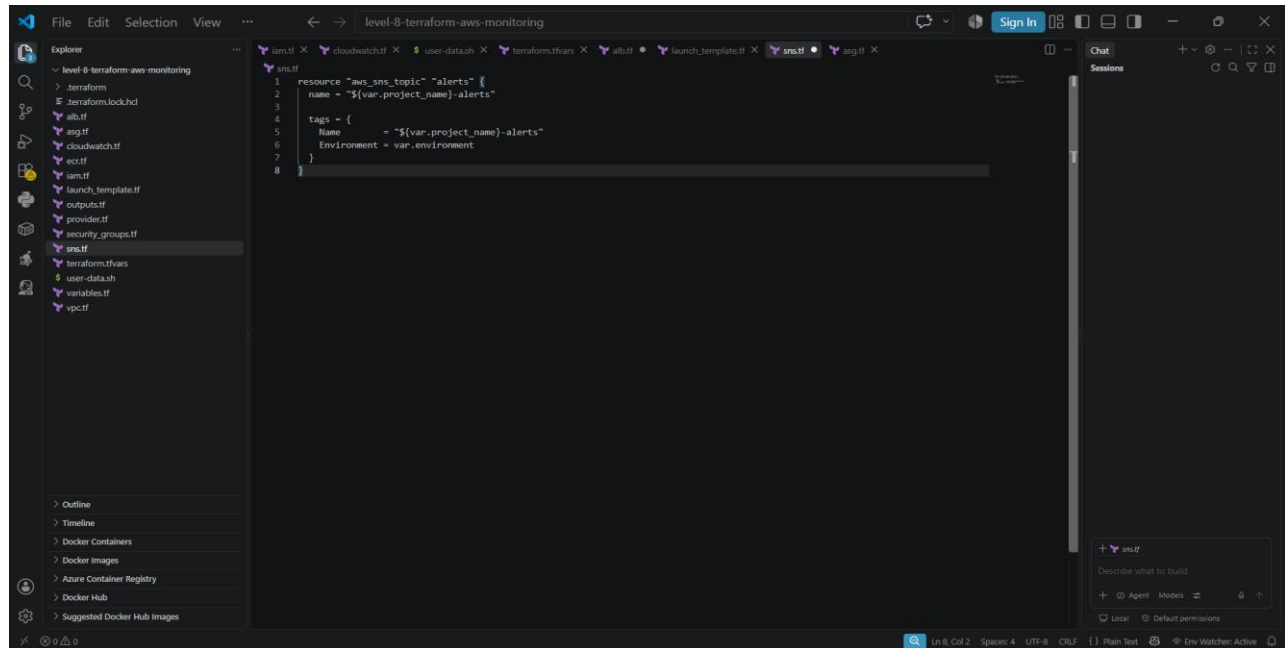


Figure 29. Additional CloudWatch alarm rules.

Step 29 – ALB 5XX alarm configuration

Instruction / Action: Create the ALB 5XX CloudWatch alarm to detect server-side HTTP errors reported by the Application Load Balancer.

Expected Result: The ALB 5XX condition is monitored by CloudWatch and can trigger the SNS notification action.

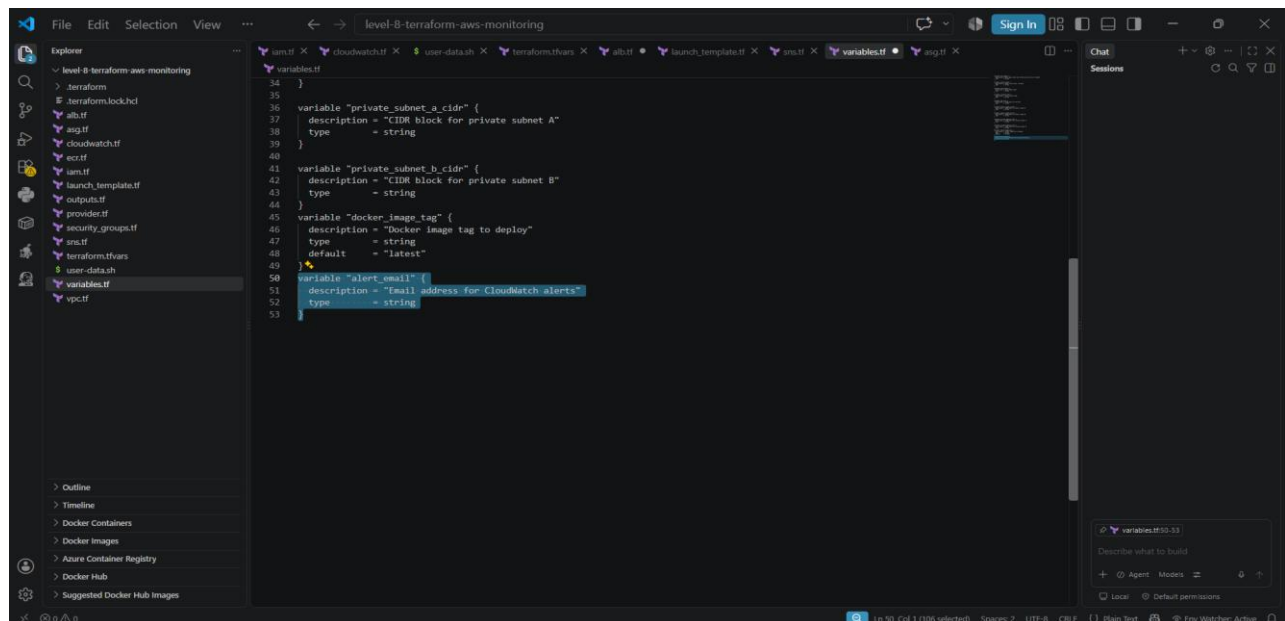


Figure 30. ALB 5XX alarm configuration.

Step 30 – Target-unhealthy and ALB response-time alarm configuration

Instruction / Action: Create the target-unhealthy alarm and the ALB response-time alarm so the monitoring layer also covers target health and application latency.

Expected Result: Target health and load-balancer response-time conditions are represented in the monitoring configuration.

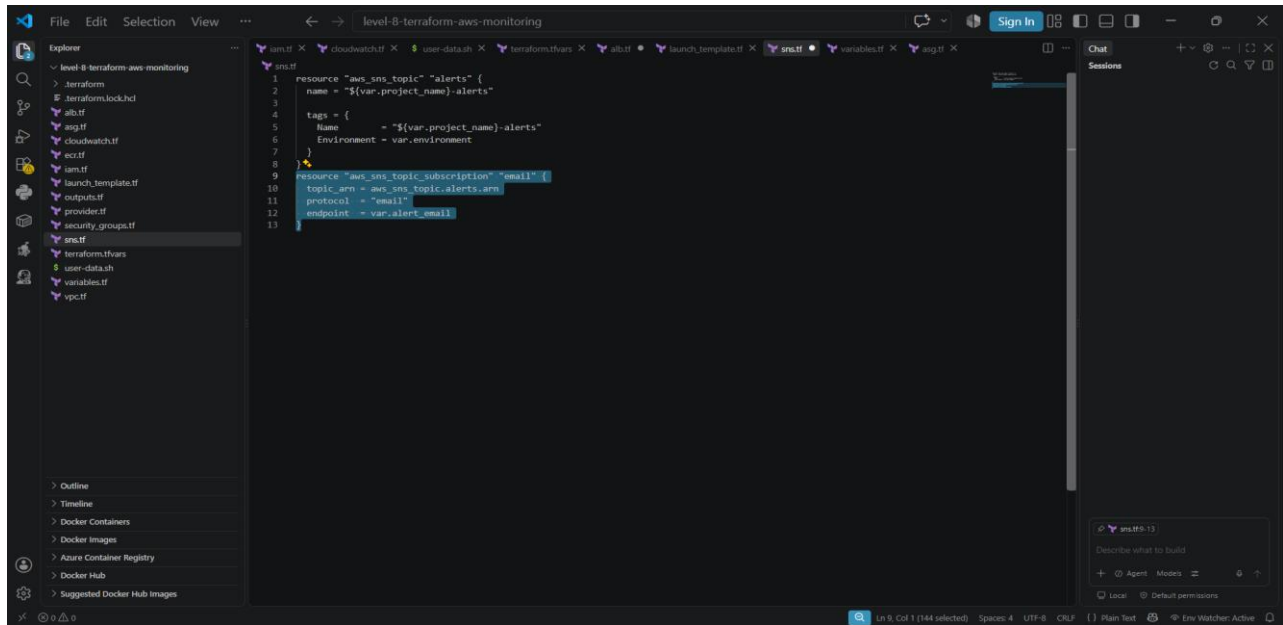


Figure 31. Target-unhealthy and ALB response-time alarm configuration.

Step 31 – Terraform alarm definitions / final alarm rules

Instruction / Action: Review the final alarm definitions together and ensure the alarm resources use the intended metrics, dimensions, thresholds and SNS actions.

Expected Result: The completed CloudWatch alarm configuration is consistent with the Level 8 monitoring design.

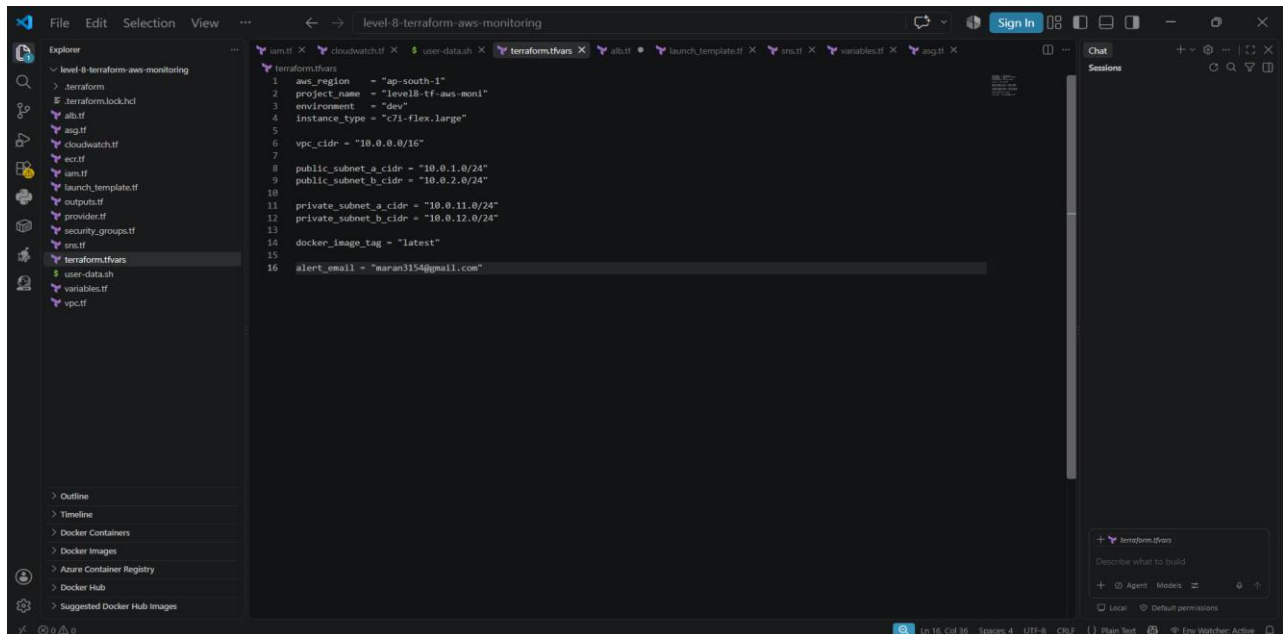
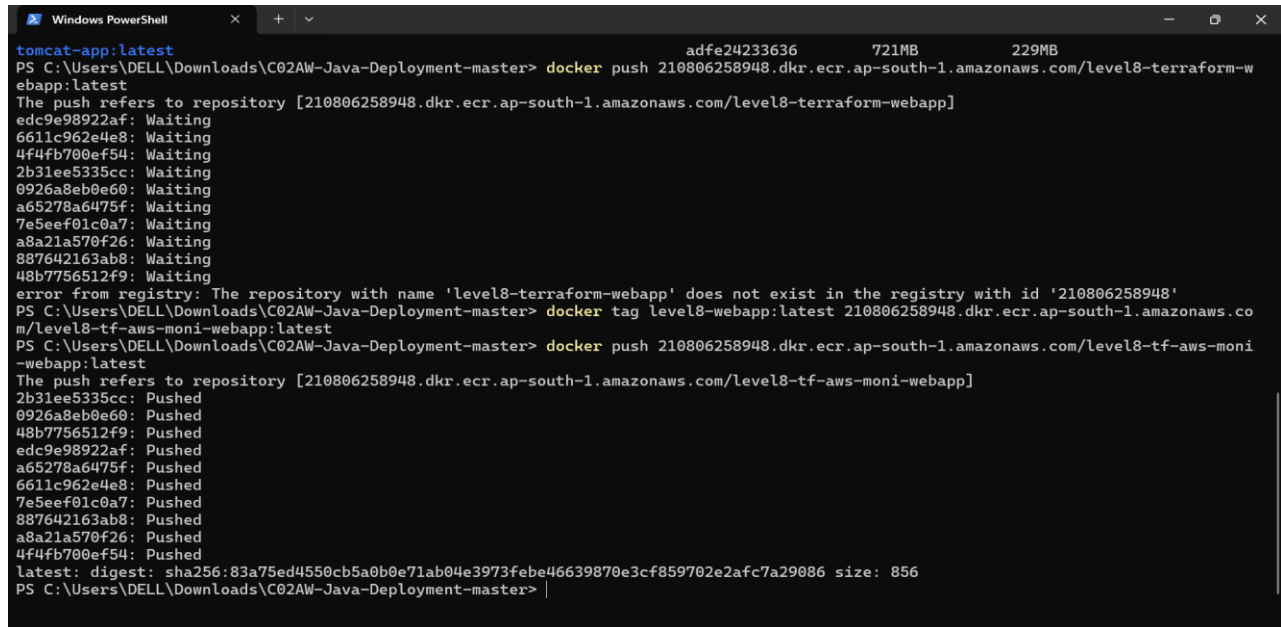


Figure 32. Terraform alarm definitions / final alarm rules.

Step 32 – CloudWatch dashboard configuration

Instruction / Action: Create the CloudWatch dashboard and add operational widgets for EC2 CPU utilization, ALB request count, target response time and unhealthy targets.

Expected Result: The dashboard provides a consolidated operational view of the application environment.



```

tomcat-app:latest
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker push 210806258948.dkr.ecr.ap-south-1.amazonaws.com/level8-terraform-w
ebapp:latest
The push refers to repository [210806258948.dkr.ecr.ap-south-1.amazonaws.com/level8-terraform-webapp]
edc9e98922af: Waiting
6611c962e4e8: Waiting
4f4fb700ef54: Waiting
2b31ee5335cc: Waiting
0926a8eb0e60: Waiting
a65278a6475f: Waiting
7e5eef01c0a7: Waiting
a8a21a570f26: Waiting
887642163ab8: Waiting
48b7756512f9: Waiting
error from registry: The repository with name 'level8-terraform-webapp' does not exist in the registry with id '210806258948'
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker tag level8-webapp:latest 210806258948.dkr.ecr.ap-south-1.amazonaws.co
m/level8-tf-aws-moni-webapp:latest
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master> docker push 210806258948.dkr.ecr.ap-south-1.amazonaws.com/level8-tf-aws-moni
-webapp:latest
The push refers to repository [210806258948.dkr.ecr.ap-south-1.amazonaws.com/level8-tf-aws-moni-webapp]
2b31ee5335cc: Pushed
0926a8eb0e60: Pushed
48b7756512f9: Pushed
edc9e98922af: Pushed
a65278a6475f: Pushed
6611c962e4e8: Pushed
7e5eef01c0a7: Pushed
887642163ab8: Pushed
a8a21a570f26: Pushed
4f4fb700ef54: Pushed
latest: digest: sha256:83a75ed4550cb5a0b0e71ab04e3973febe46639870e3cf859702e2afc7a29086 size: 856
PS C:\Users\DELL\Downloads\C02AW-Java-Deployment-master>

```

Figure 33. CloudWatch dashboard configuration.

Step 33 – Docker image push to ECR; initial repository mismatch and corrected tag

Instruction / Action: Build/tag the Dockerized Java/Tomcat application and push it to ECR. If the repository URI does not match the image tag, correct the repository name/URI and repeat the push using the correct ECR repository.

Expected Result: The application image is successfully available in the intended private ECR repository. The supplied evidence records the initial repository-name mismatch and subsequent correction.

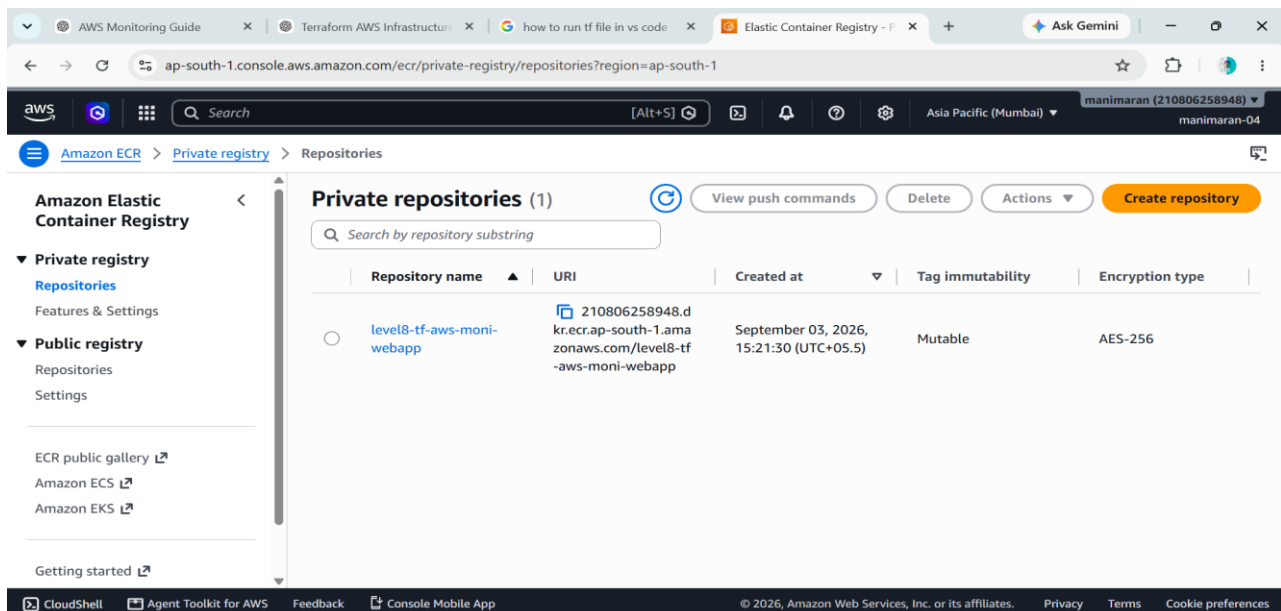


Figure 34. Docker image push to ECR; initial repository mismatch and corrected tag.

Step 34 – ECR private repository visible in AWS console

Instruction / Action: Open the Amazon ECR private repositories view and verify that level8-tf-aws-moni-webapp exists.

Expected Result: The private ECR repository is visible in AWS and is available for application image storage.

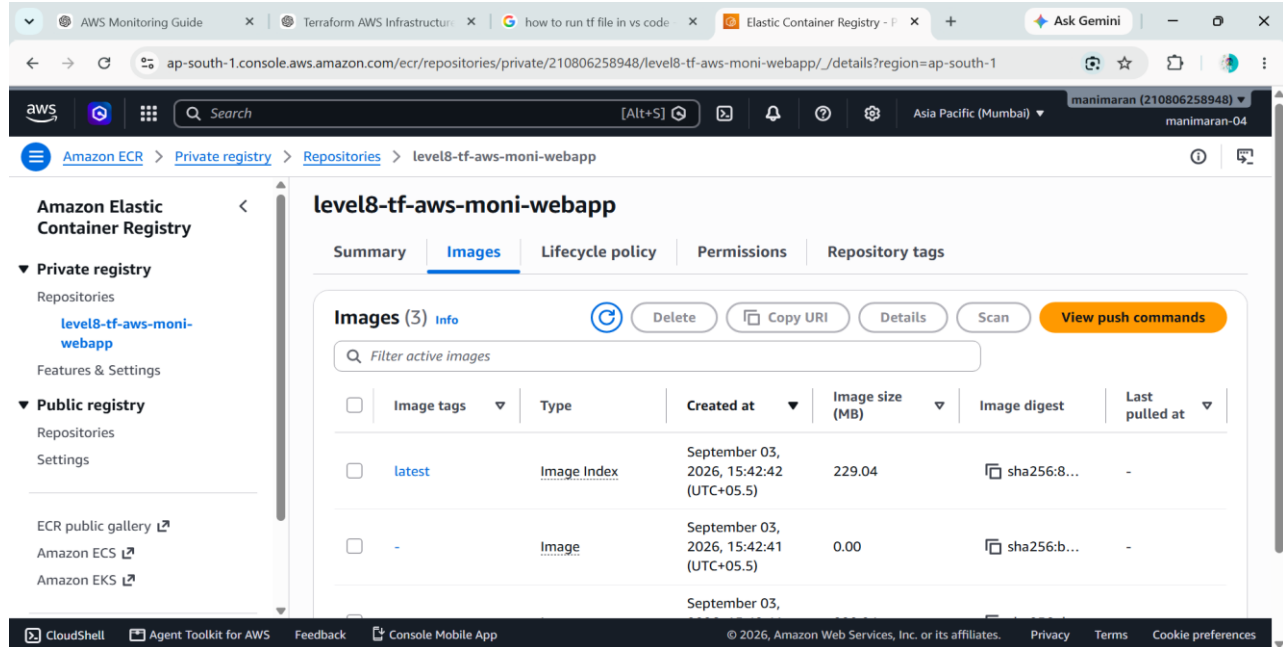


Figure 35. ECR private repository visible in AWS console.

Step 35 – ECR repository details / image information

Instruction / Action: Open the ECR repository and inspect its image list/details to verify that the expected Docker image and tag are present.

Expected Result: The repository contains the application image required by the EC2 bootstrap process.

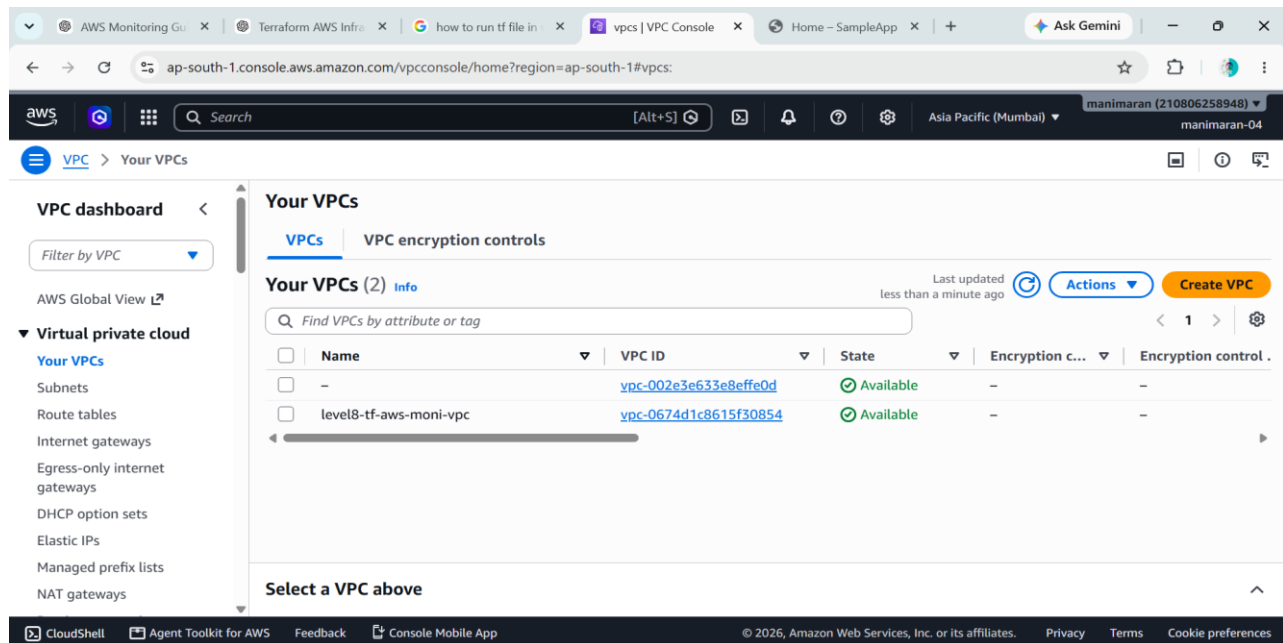


Figure 36. ECR repository details / image information.

Step 36 – AWS console resource verification

Instruction / Action: Open the VPC console and verify that the Terraform-managed project VPC exists in ap-south-1.

Expected Result: The VPC created by the Terraform configuration is visible and available.

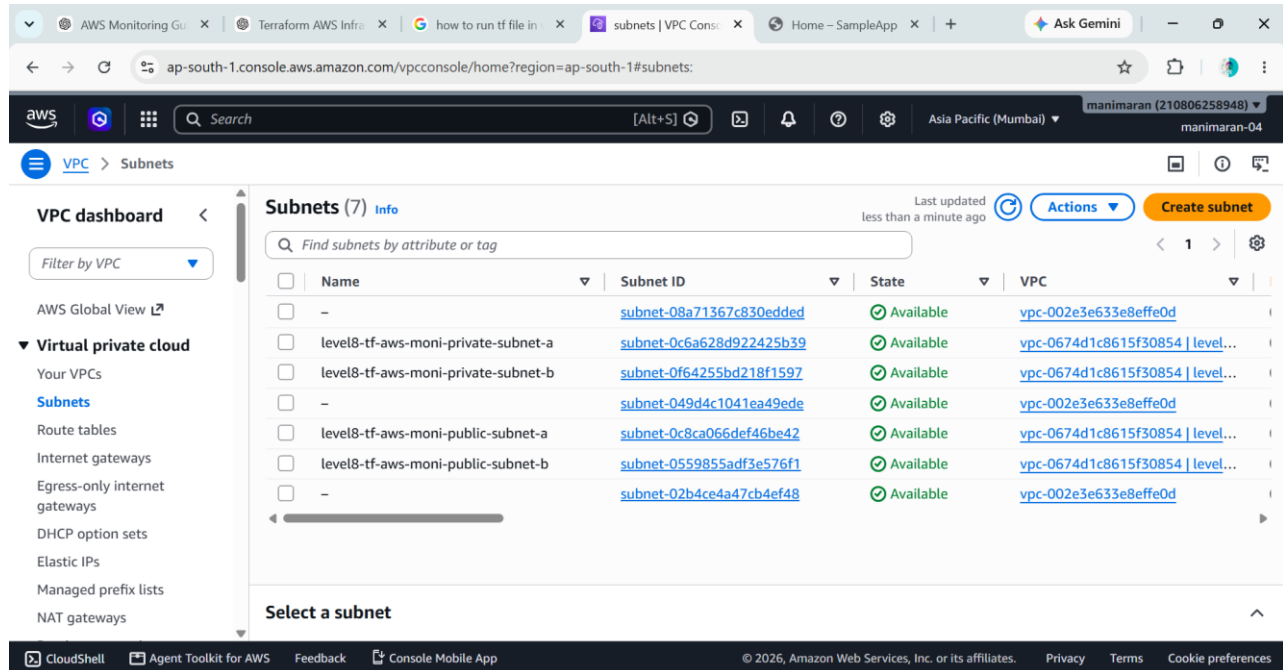


Figure 37. AWS console resource verification.

Step 37 – AWS console resource verification

Instruction / Action: Open the Subnets view and verify the project public and private subnets and their VPC associations.

Expected Result: The expected subnet resources are visible and associated with the project VPC.

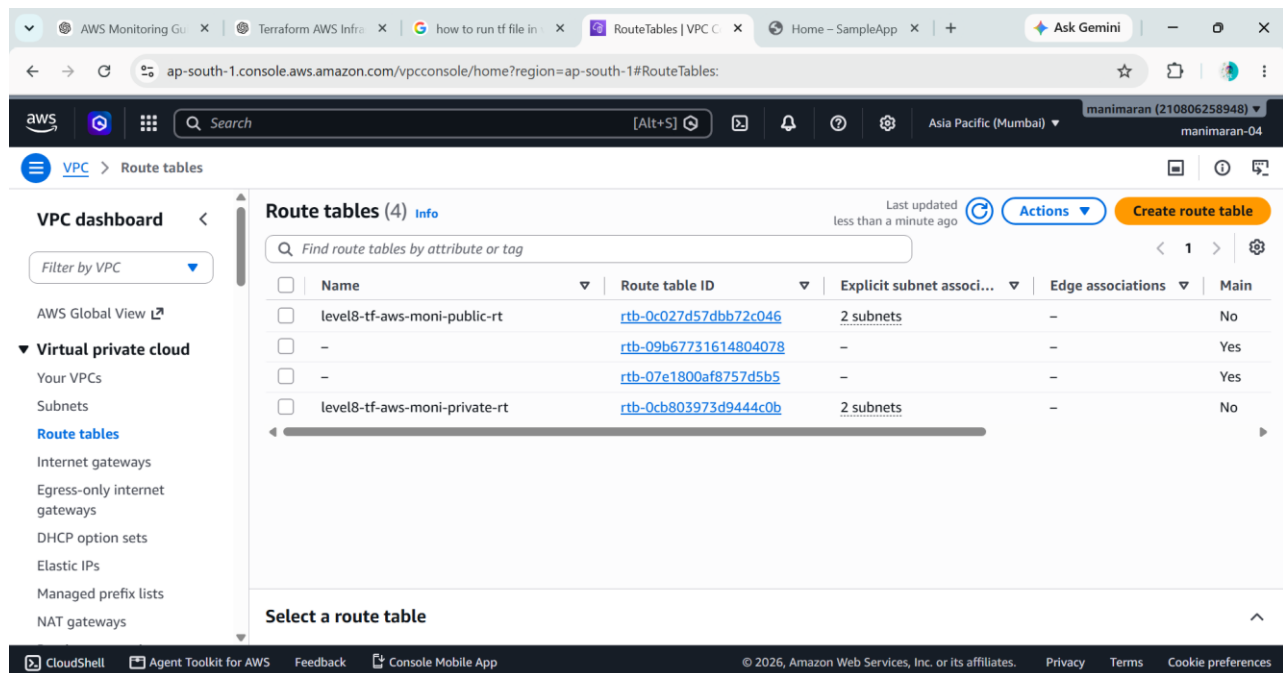


Figure 38. AWS console resource verification.

Step 38 – AWS console networking/resource details

Instruction / Action: Open Route Tables and verify the Terraform-created route tables and subnet associations for the project network.

Expected Result: The route tables and their associations are visible in the AWS console.

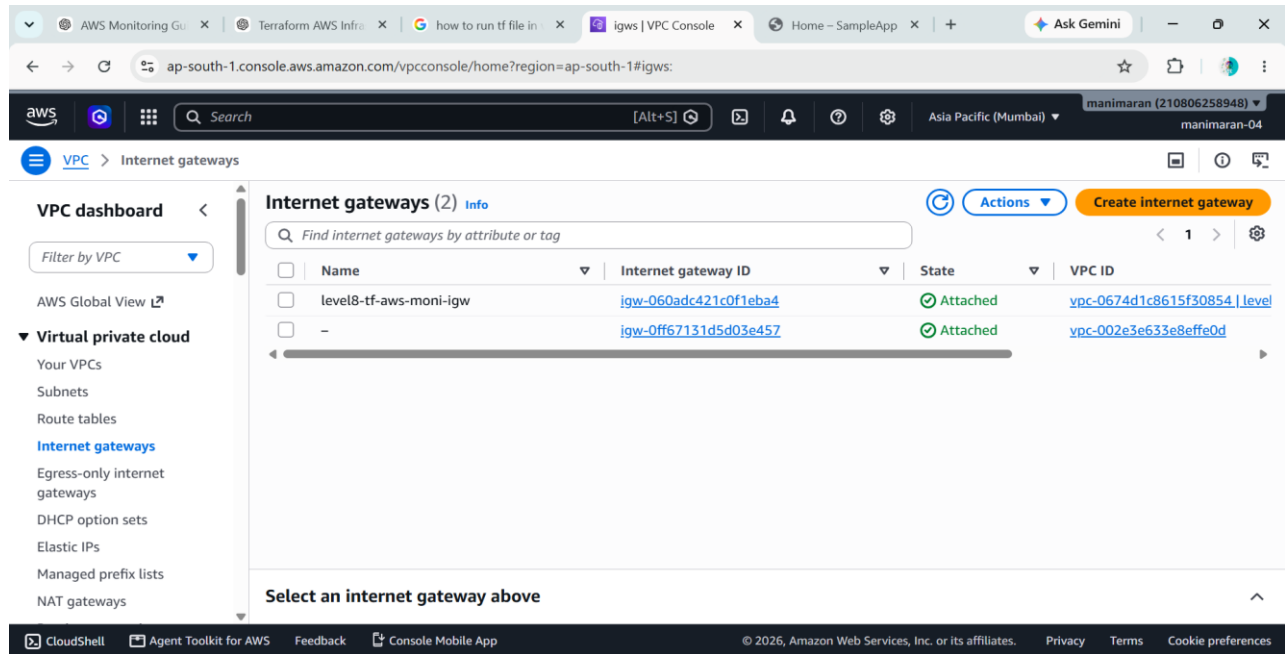


Figure 39. AWS console networking/resource details.

Step 39 – AWS console IAM/ECR configuration evidence

Instruction / Action: Open Internet Gateways and NAT Gateways and verify that the project Internet Gateway is attached and the NAT Gateway is available.

Expected Result: The required Internet and NAT connectivity resources are present.

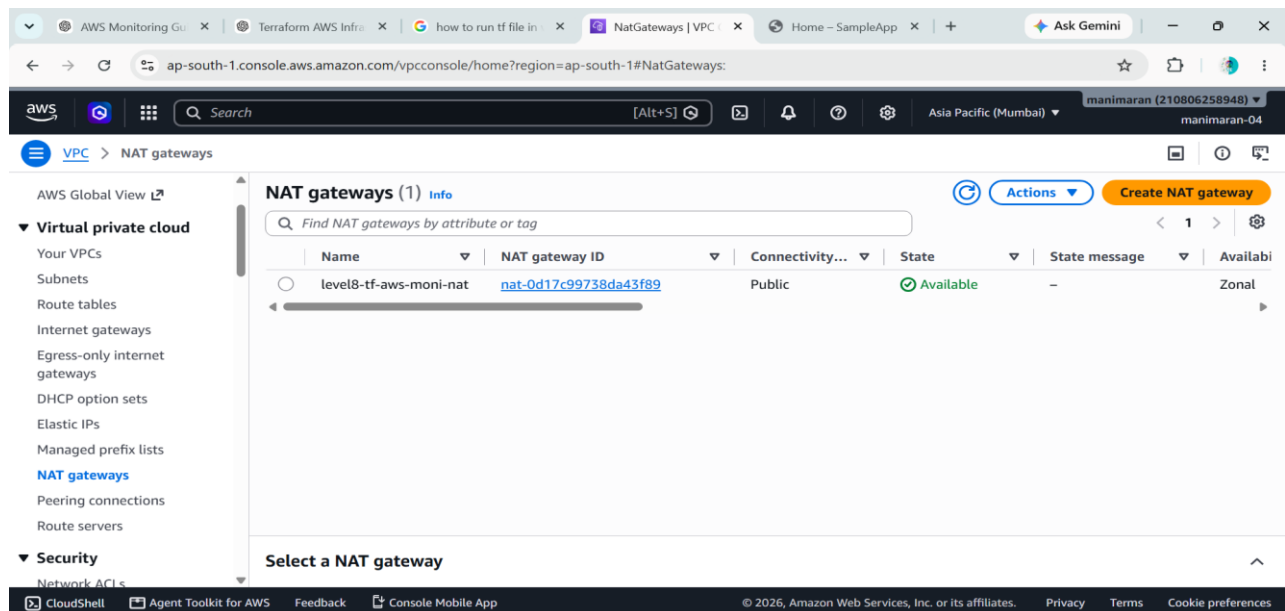


Figure 40. AWS console IAM/ECR configuration evidence.

Step 40 – AWS console ECR/resource details

Instruction / Action: Open the Load Balancers view and verify that the Terraform-created Application Load Balancer is active.

Expected Result: The internet-facing ALB is present and available to receive application traffic.

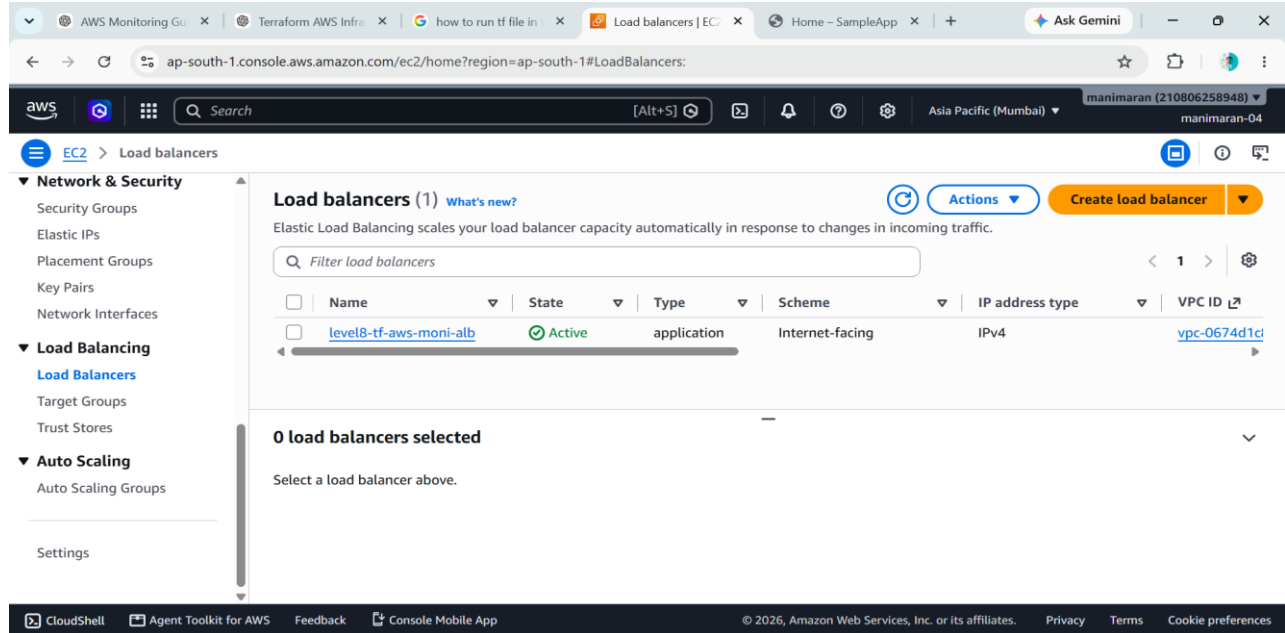


Figure 41. AWS console ECR/resource details.

Step 41 – AWS console resource status evidence

Instruction / Action: Open Target Groups and verify the application target group, its HTTP port 8080 configuration and target registration.

Expected Result: The target group is present and ready to route traffic to the application instances.

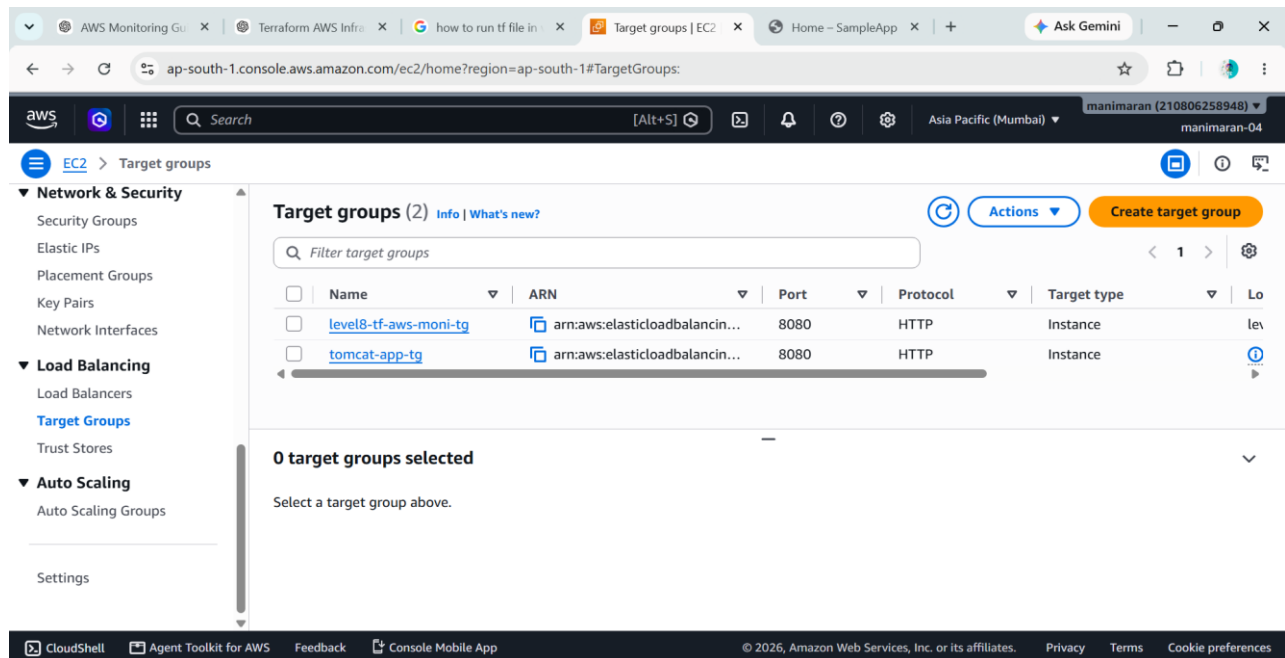


Figure 42. AWS console resource status evidence.

Step 42 – AWS console resource status evidence

Instruction / Action: Open Auto Scaling Groups and verify the application ASG, its launch-template association and configured capacity.

Expected Result: The application Auto Scaling Group is visible and uses the intended launch template.

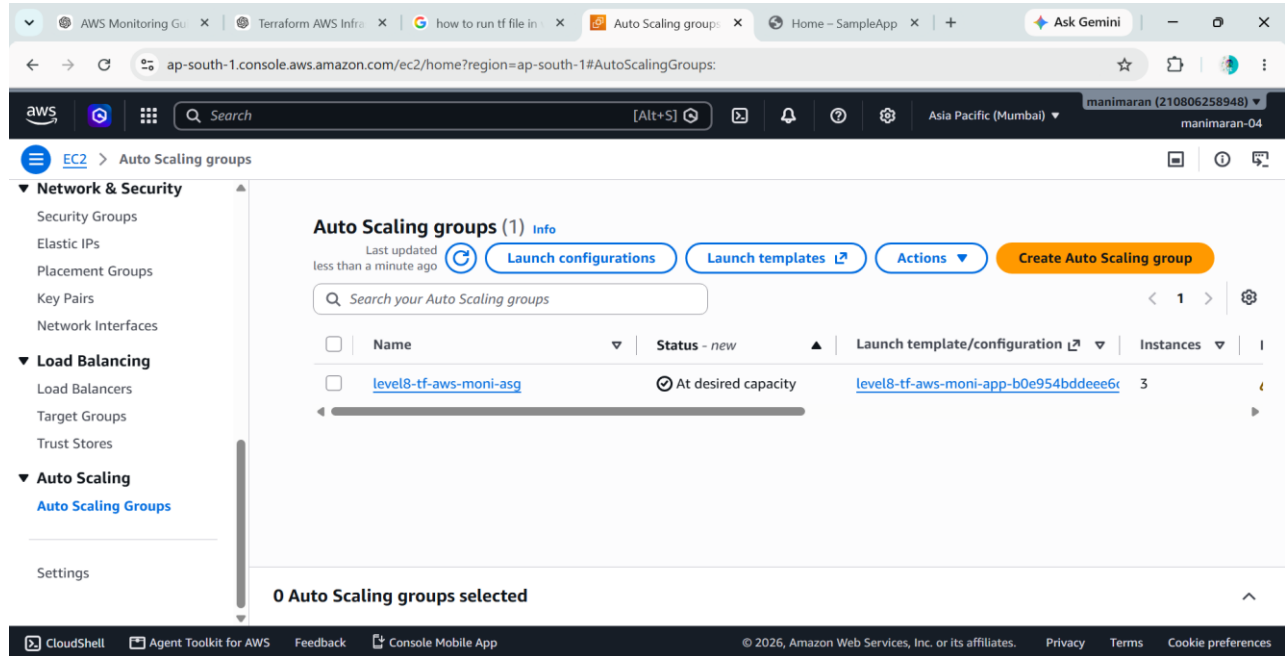


Figure 43. AWS console resource status evidence.

Step 43 – AWS console resource status evidence

Instruction / Action: Open Elastic IPs and verify the public address associated with the NAT Gateway.

Expected Result: The NAT Gateway has the expected public Elastic IP association.

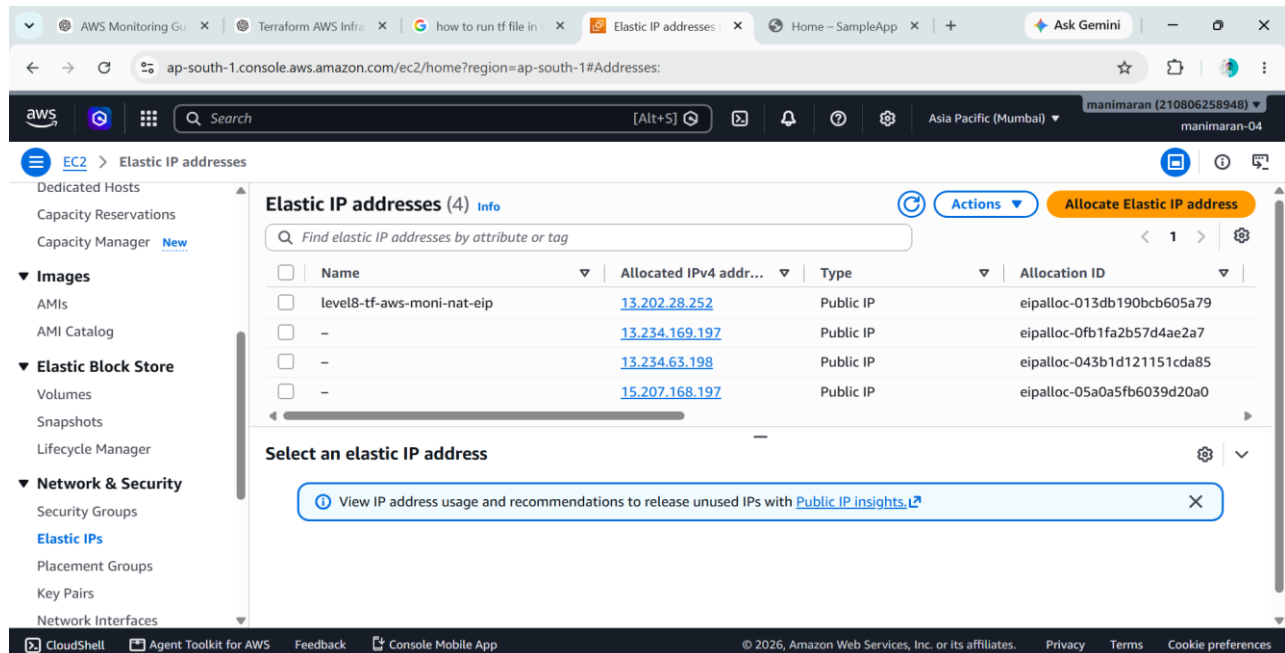


Figure 44. AWS console resource status evidence.

Step 44 – AWS console resource status evidence

Instruction / Action: Open the EC2 Instances view and verify that the application instances have launched from the Terraform-managed ASG.

Expected Result: The application EC2 instances are running and are part of the deployed environment.

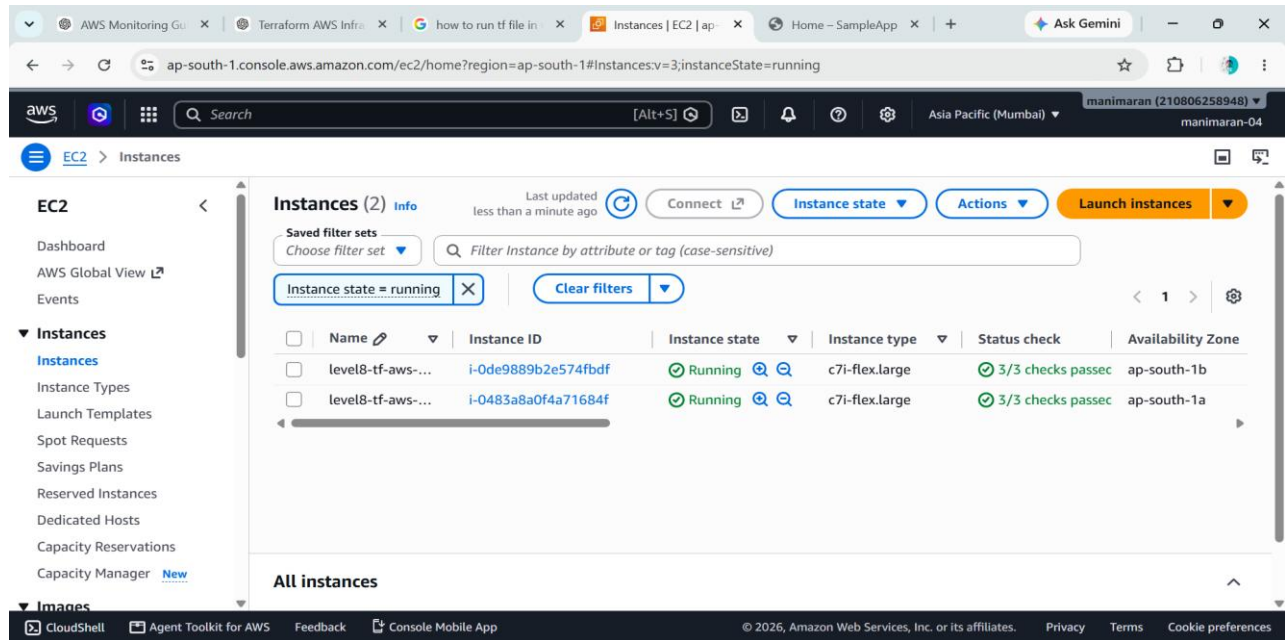


Figure 45. AWS console resource status evidence.

Step 45 – AWS console configuration evidence

Instruction / Action: Open the application through the ALB/browser endpoint and verify that the Java web application home page loads successfully.

Expected Result: The application is reachable through the load balancer and the web page is served successfully.

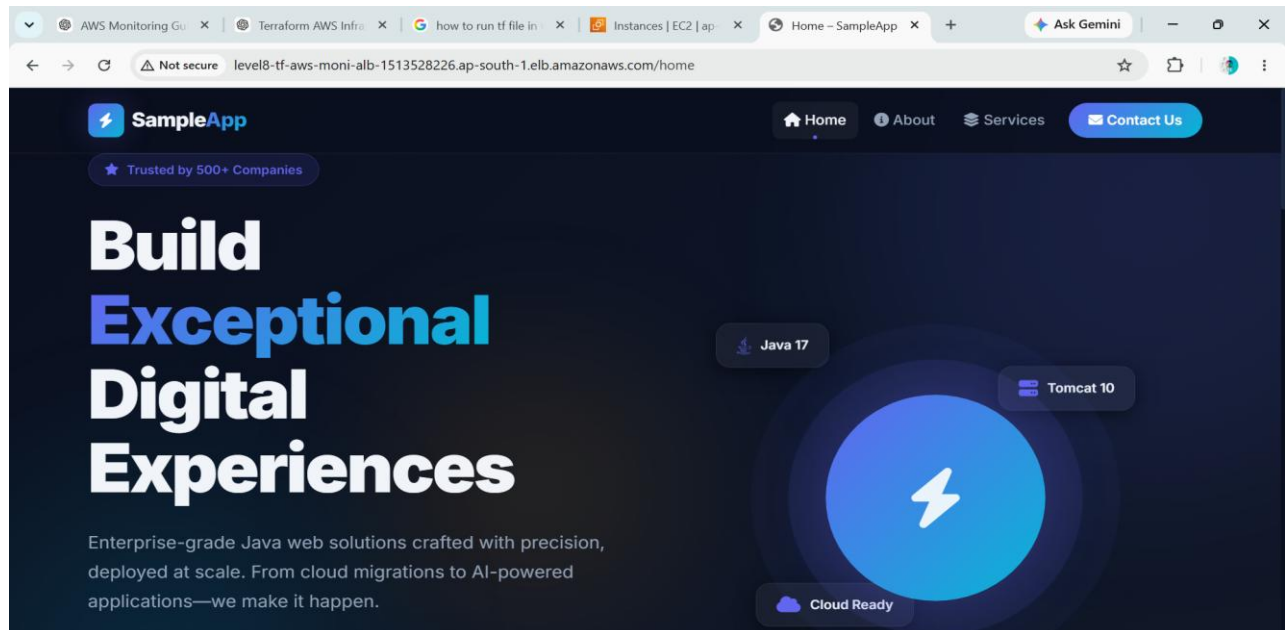


Figure 46. AWS console configuration evidence.

Step 46 – AWS console/browser verification evidence

Instruction / Action: Open the AWS notification/subscription verification view and confirm the SNS subscription/notification setup used by the monitoring stack.

Expected Result: The notification subscription is available for alarm delivery.

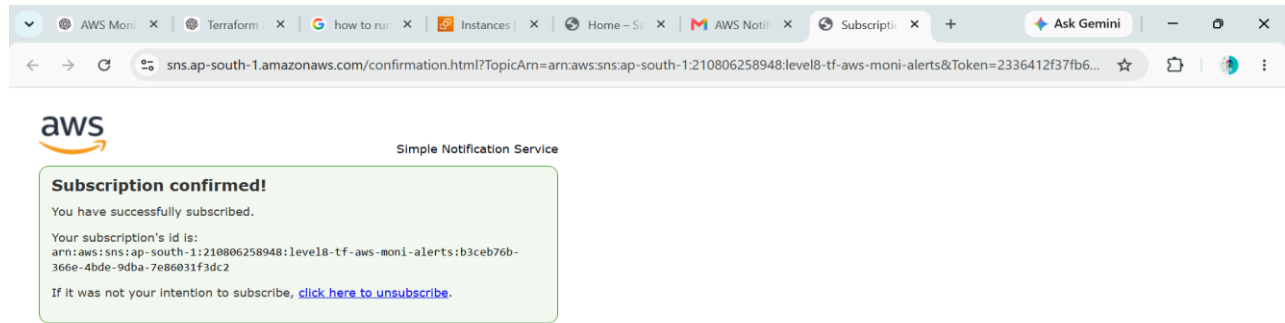


Figure 47. AWS console/browser verification evidence.

Step 47 – SNS subscription confirmation page

Instruction / Action: Complete the SNS subscription confirmation process and verify that the subscription is confirmed before relying on alarm notifications.

Expected Result: The SNS email subscription is confirmed and ready to receive CloudWatch alarm messages.

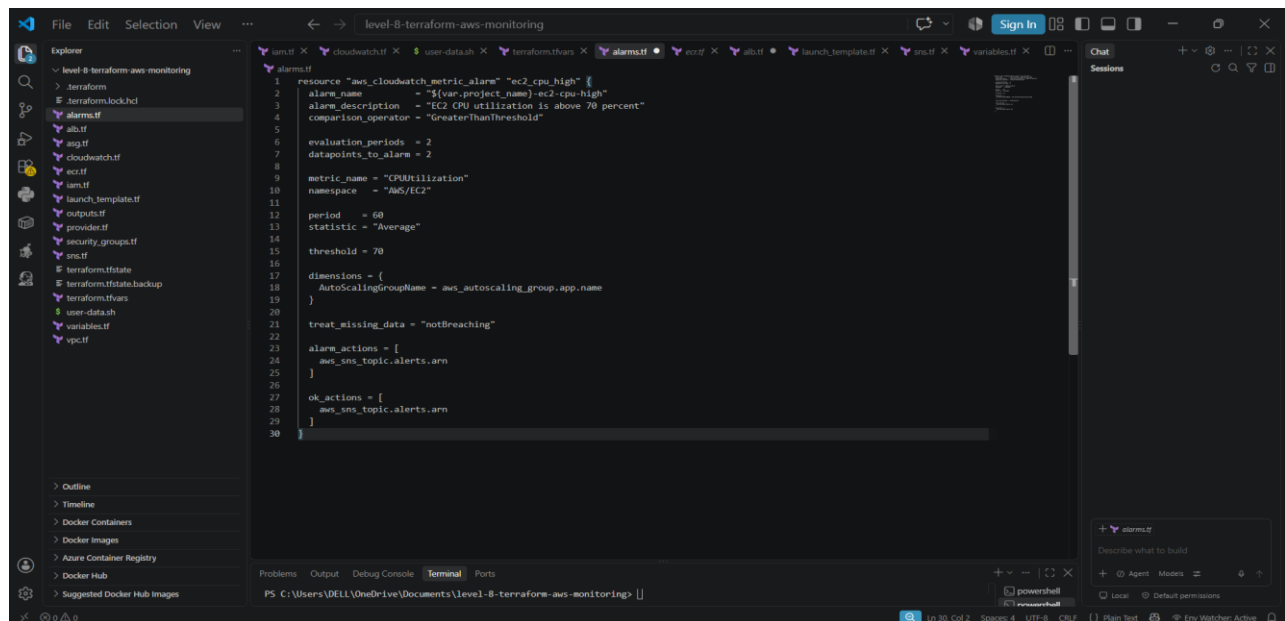


Figure 48. SNS subscription confirmation page.

Step 48 – Terraform user-data / CloudWatch bootstrap configuration

Instruction / Action: Review the Terraform user-data script and verify that EC2 bootstrap installs/configures the CloudWatch Agent and prepares the monitoring environment.

Expected Result: The user-data script contains the required CloudWatch bootstrap commands.

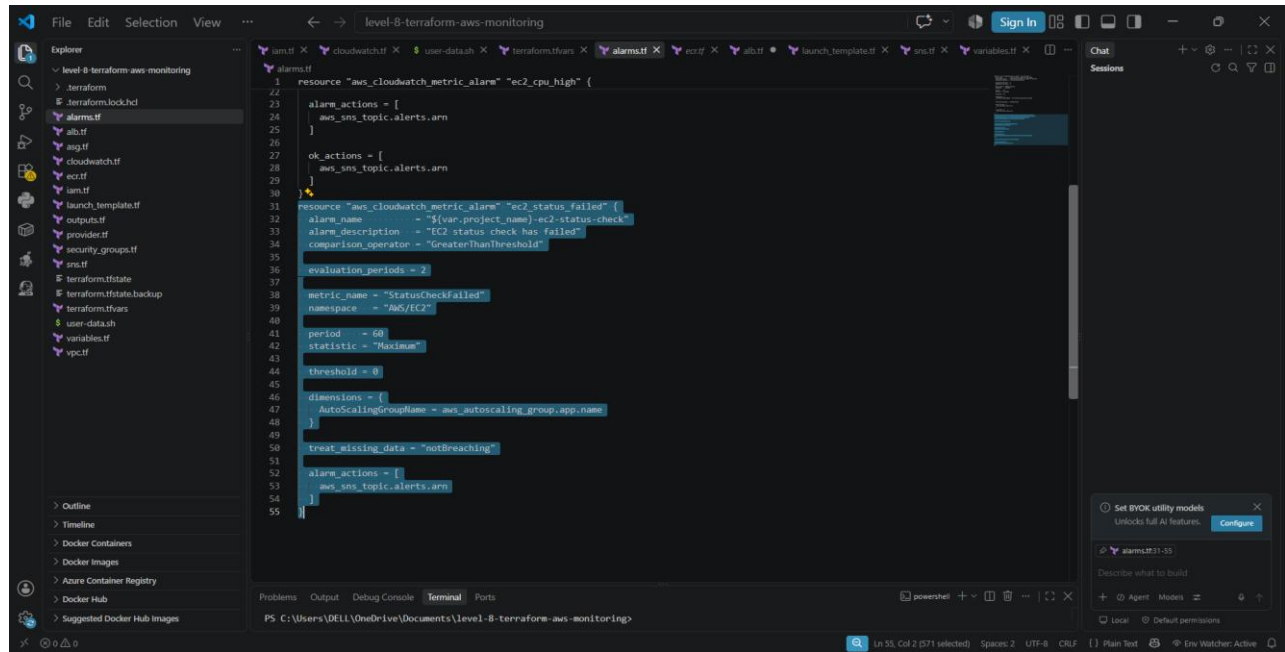


Figure 49. Terraform user-data / CloudWatch bootstrap configuration.

Step 49 – Terraform user-data and monitoring configuration

Instruction / Action: Review the monitoring portion of user-data and confirm that the CloudWatch Agent configuration is written/applied during instance startup.

Expected Result: New instances receive the intended monitoring configuration automatically.

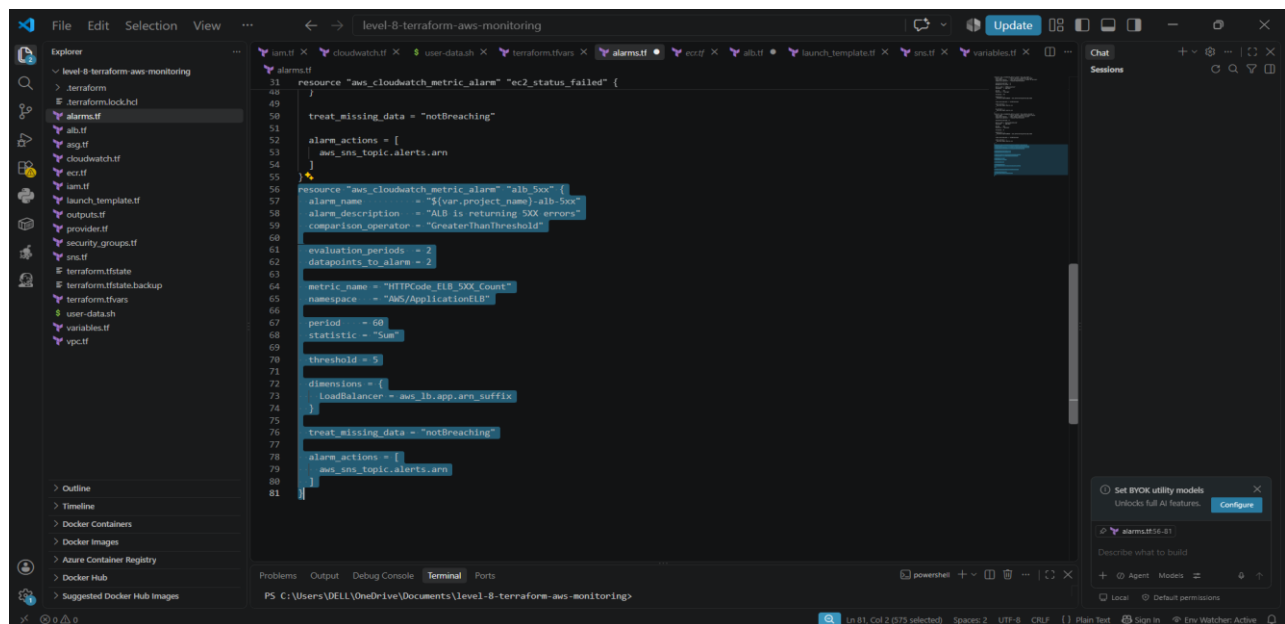


Figure 50. Terraform user-data and monitoring configuration.

Step 50 – Terraform application bootstrap configuration

Instruction / Action: Review the application bootstrap commands that authenticate to ECR, pull the selected image and start the Docker/Tomcat container on port 8080.

Expected Result: EC2 user data contains the application startup sequence required to run the Java web application.

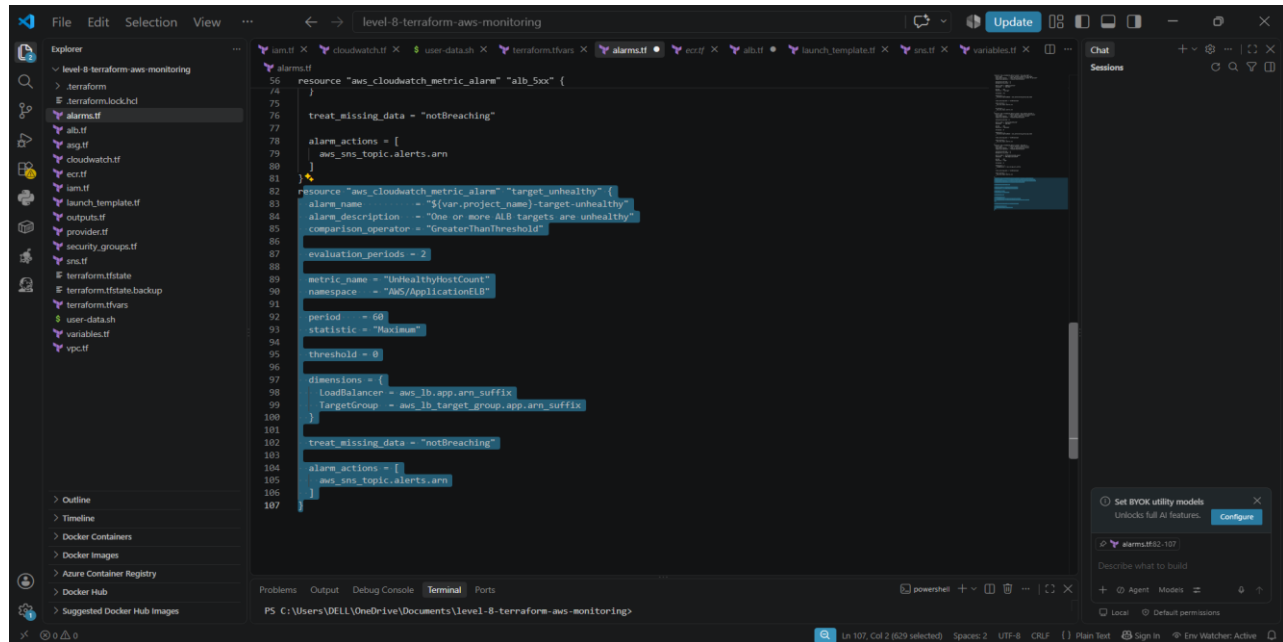


Figure 51. Terraform application bootstrap configuration.

Step 51 – Terraform launch-template/monitoring configuration

Instruction / Action: Review the launch-template and monitoring configuration together to verify that IAM, monitoring and application bootstrap are passed to each instance.

Expected Result: The launch template provides a complete startup environment for the application fleet.

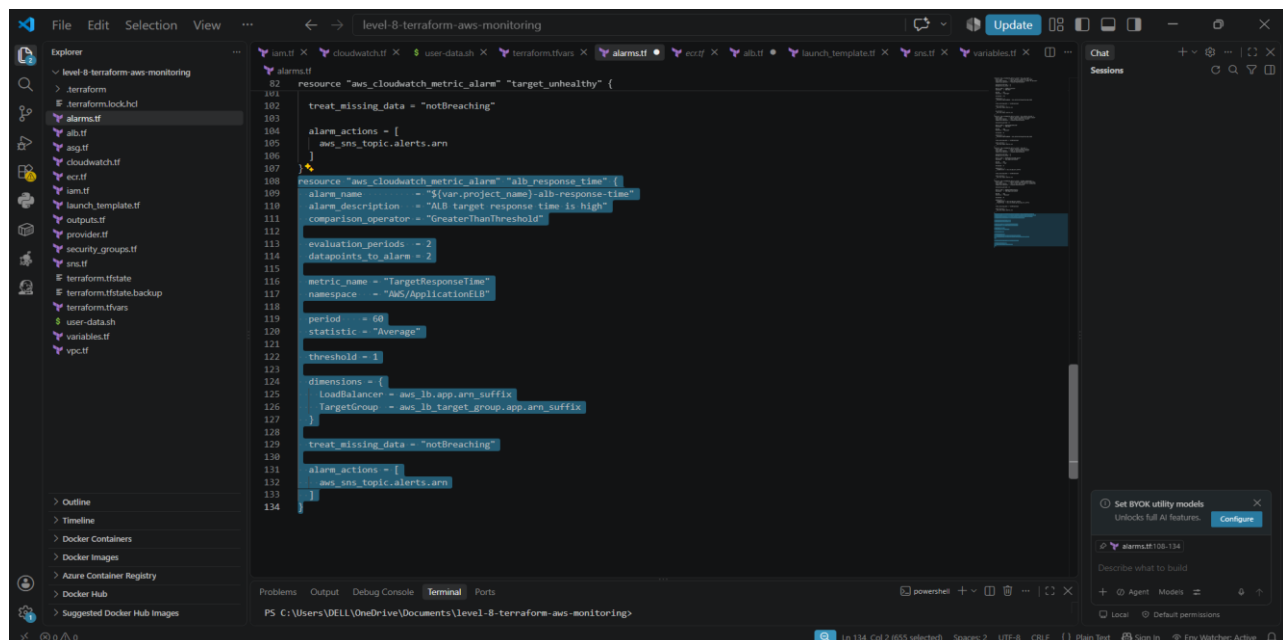


Figure 52. Terraform launch-template/monitoring configuration.

Step 52 – Terraform monitoring configuration

Instruction / Action: Review the Terraform monitoring configuration and verify that the CloudWatch Agent/logging resources are connected to the EC2 environment.

Expected Result: The monitoring resources are linked to the infrastructure being provisioned.

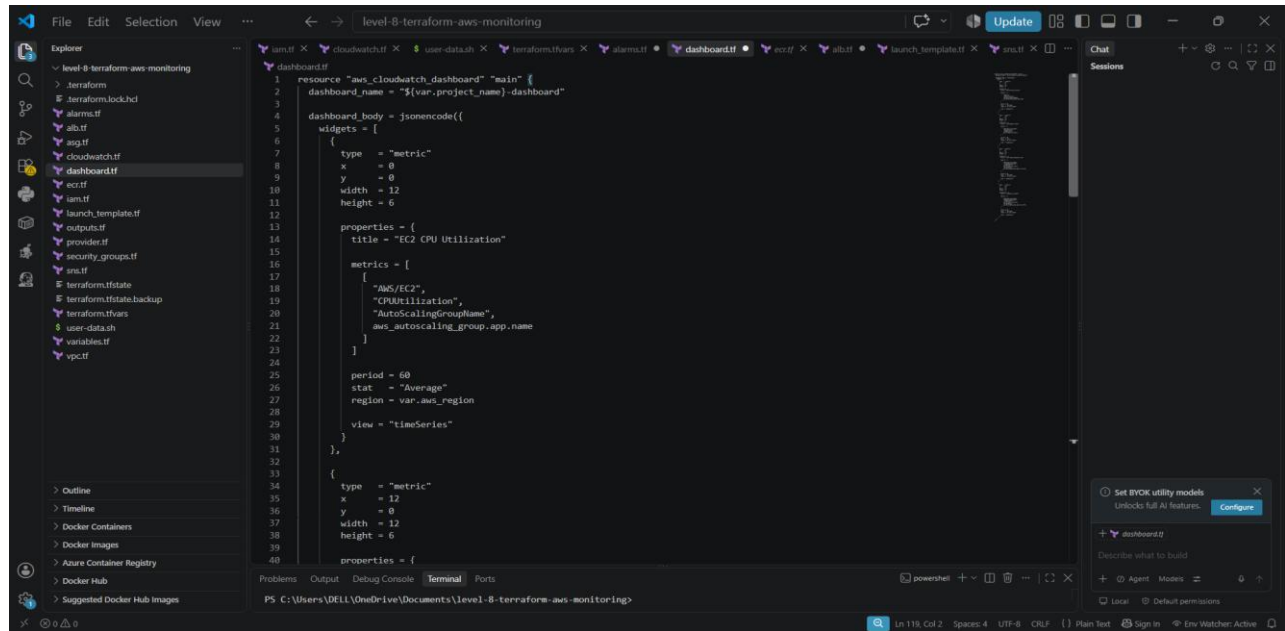


Figure 53. Terraform monitoring configuration.

Step 53 – Terraform CloudWatch configuration

Instruction / Action: Review the Terraform CloudWatch resource definitions and confirm that the log group and monitoring resources use the project naming and references.

Expected Result: The CloudWatch infrastructure is defined consistently in Terraform.

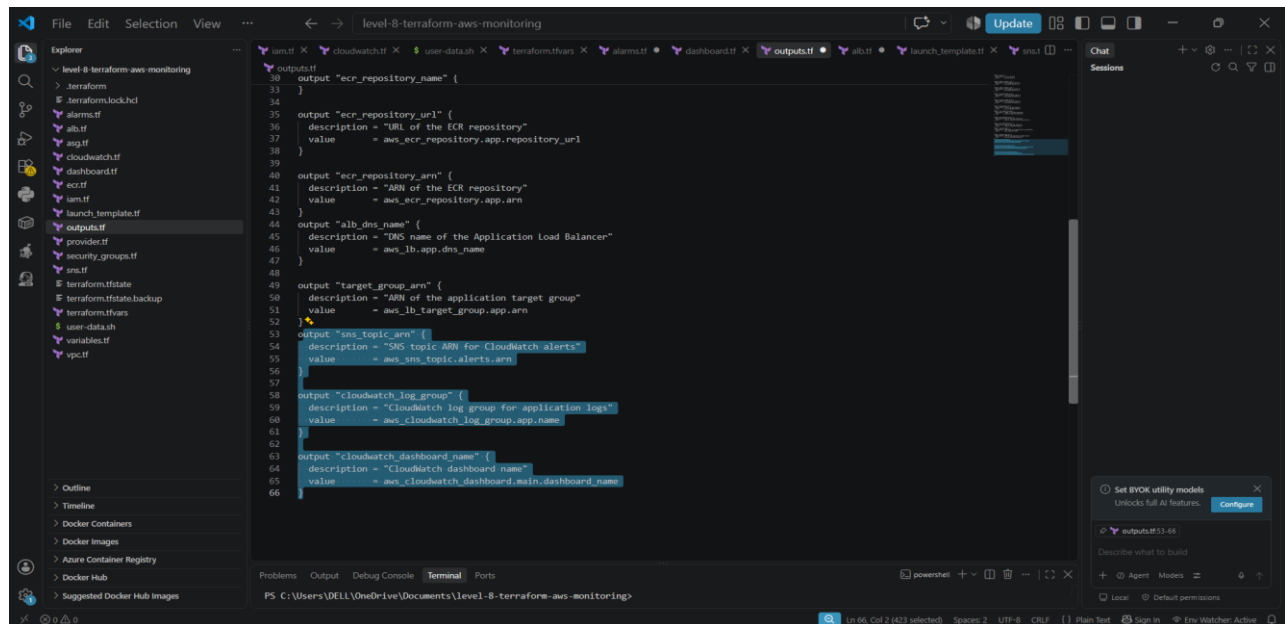


Figure 54. Terraform CloudWatch configuration.

Step 54 – Terraform alarm configuration

Instruction / Action: Review the Terraform alarm resource definitions and confirm the CPU and infrastructure-health alarm parameters and notification actions.

Expected Result: The alarm resources contain the intended thresholds, periods and SNS actions.

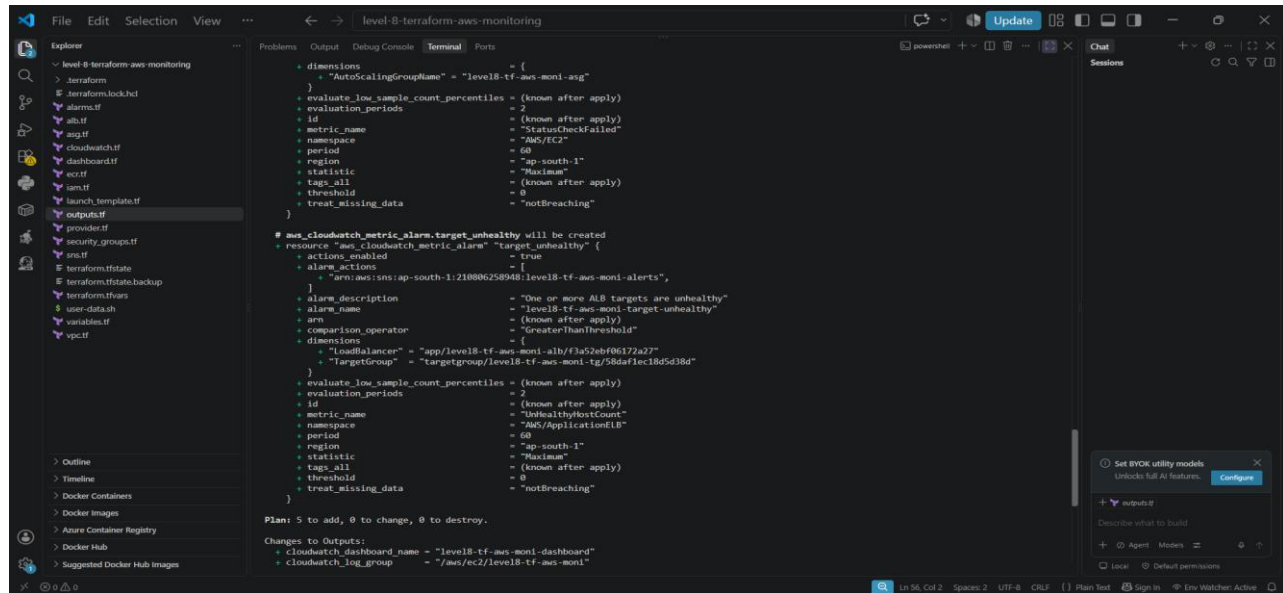


Figure 55. Terraform alarm configuration.

Step 55 – Terraform alarm configuration

Instruction / Action: Review the final alarm configuration and the Terraform execution/output associated with the monitoring stack.

Expected Result: The monitoring configuration is ready to be applied and the Terraform run reflects the defined alarm resources.

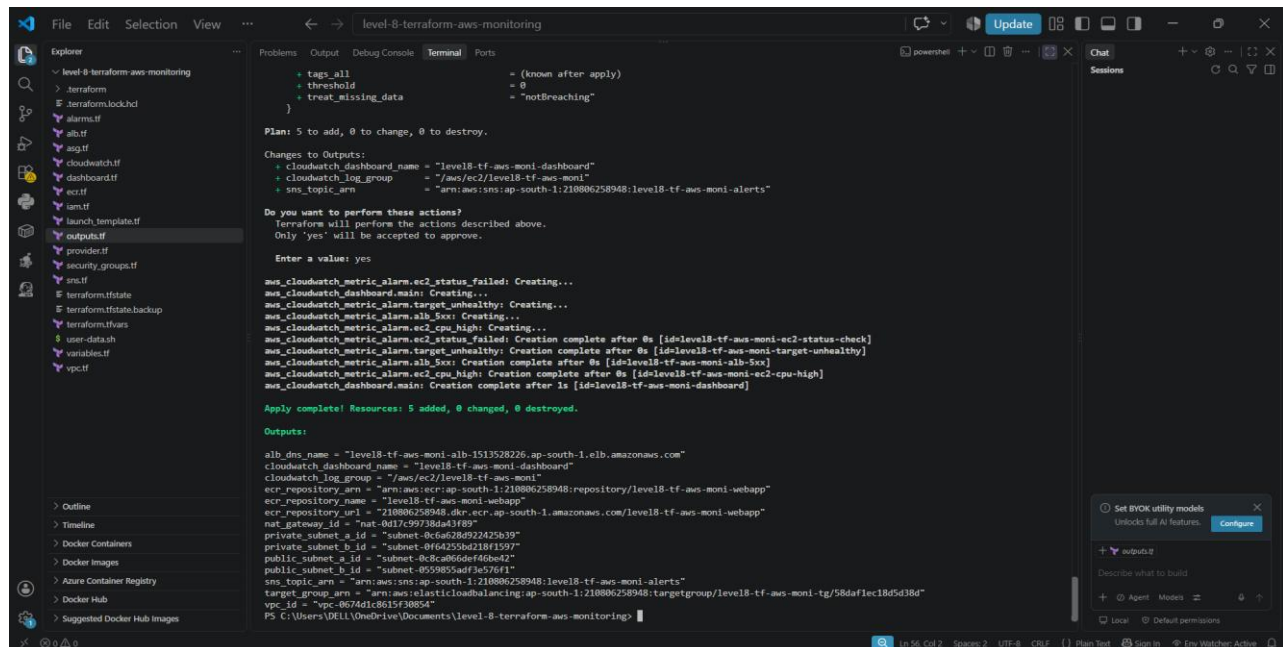


Figure 56. Terraform alarm configuration.

Step 56 – Terraform dashboard configuration

Instruction / Action: Review the Terraform CloudWatch dashboard definition and confirm the operational widgets and metrics are configured.

Expected Result: The dashboard definition contains the intended monitoring widgets.

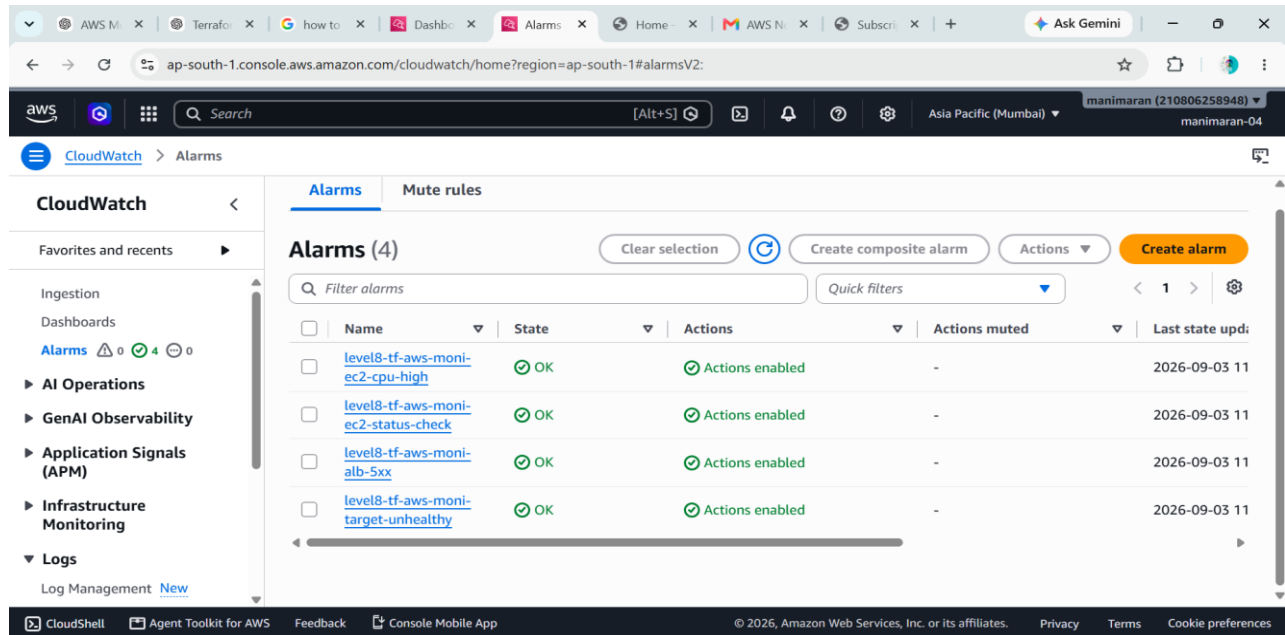


Figure 57. Terraform dashboard configuration.

Step 57 – CloudWatch alarm list showing four alarms in OK state

Instruction / Action: Open CloudWatch Alarms and verify the four supplied alarms are present and in OK state: CPU high, status check, ALB 5XX and target unhealthy.

Expected Result: The configured alarms are visible and currently show OK state in the supplied evidence.

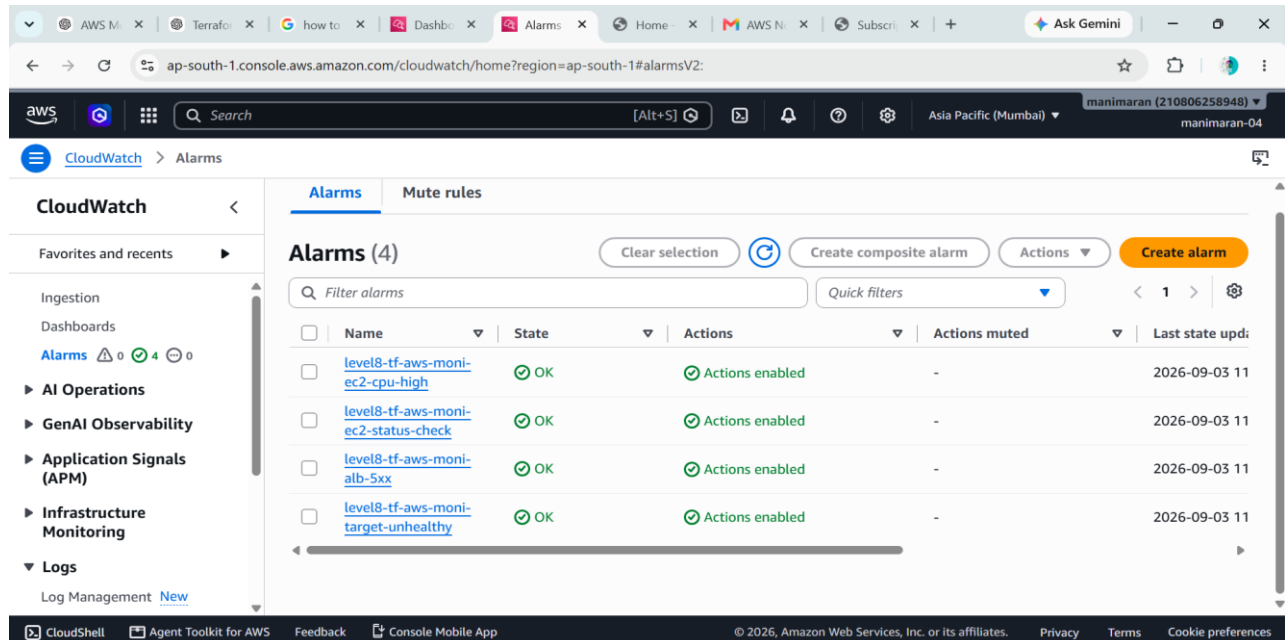


Figure 58. CloudWatch alarm list showing four alarms in OK state.

Step 58 – CloudWatch alarm monitoring verification

Instruction / Action: Open an individual CloudWatch alarm/monitoring view and verify its state and monitoring data.

Expected Result: The alarm can be inspected in CloudWatch and its state/metric information is available.

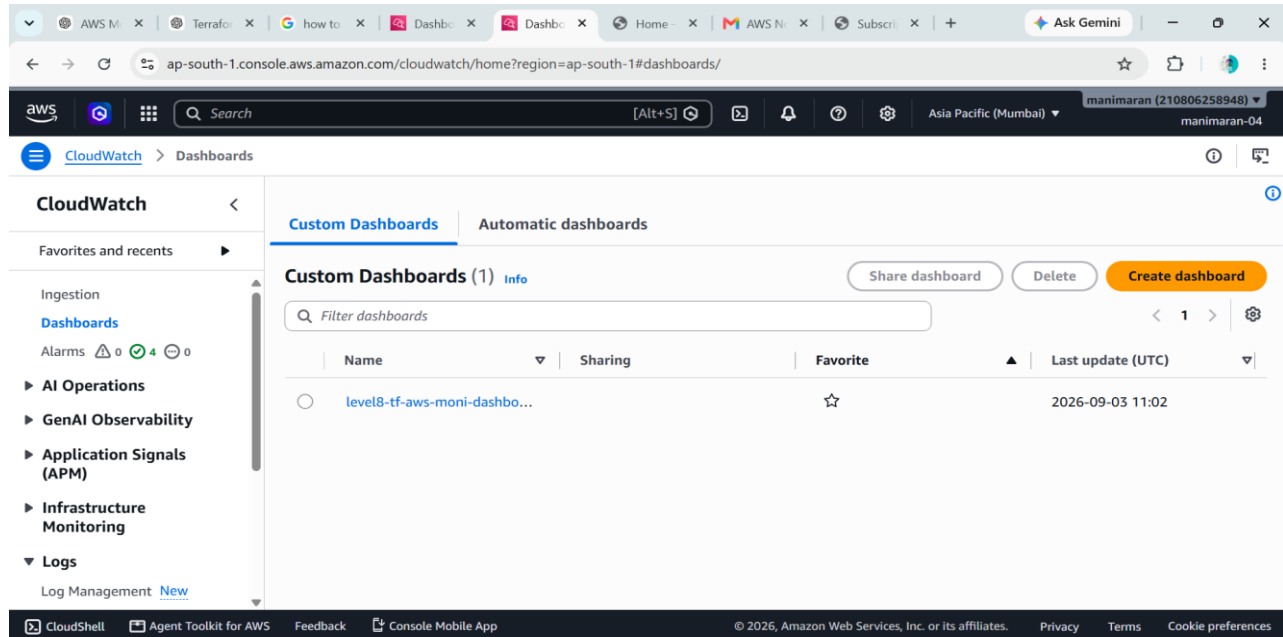


Figure 59. CloudWatch alarm monitoring verification.

Step 59 – CloudWatch dashboard list

Instruction / Action: Open CloudWatch Custom Dashboards and verify that the project dashboard level8-tf-aws-moni-dashboard exists.

Expected Result: The Terraform-created dashboard is visible in the CloudWatch dashboard list.

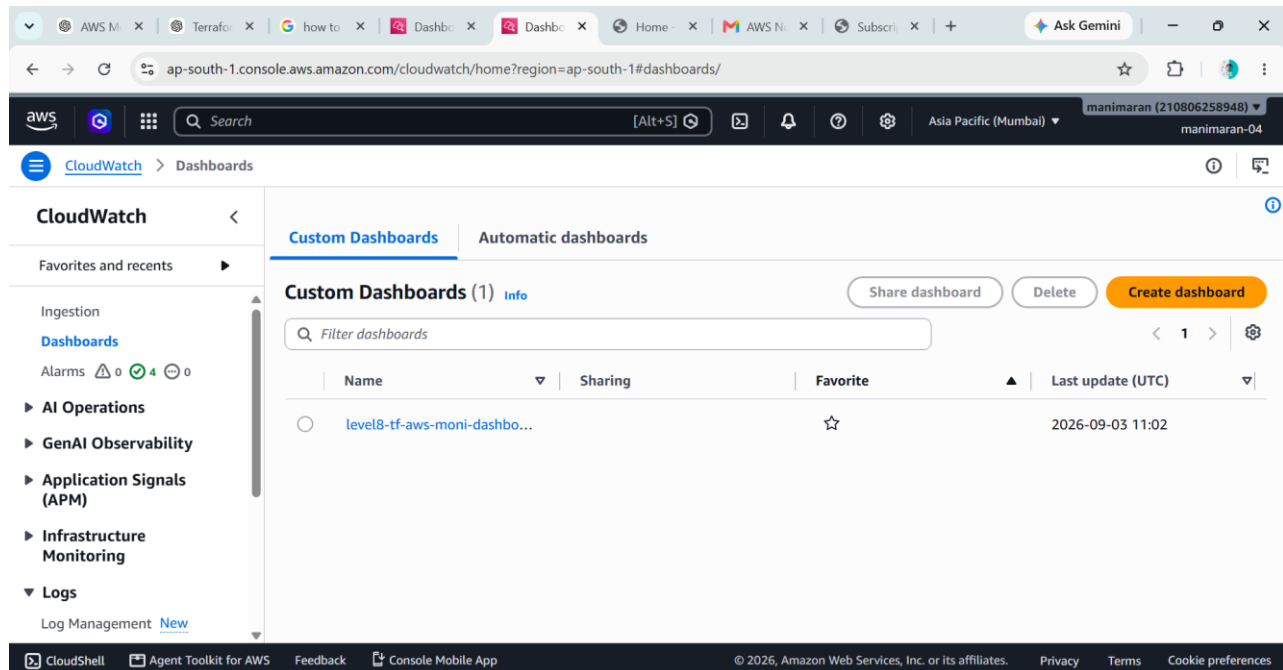


Figure 60. CloudWatch dashboard list.

Step 60 – CloudWatch dashboard with CPU, ALB request count, response time and unhealthy-target widgets

Instruction / Action: Open the project dashboard and inspect the EC2 CPU, ALB request count, target response time and unhealthy-target widgets.

Expected Result: The dashboard displays the intended operational metrics for the application stack.

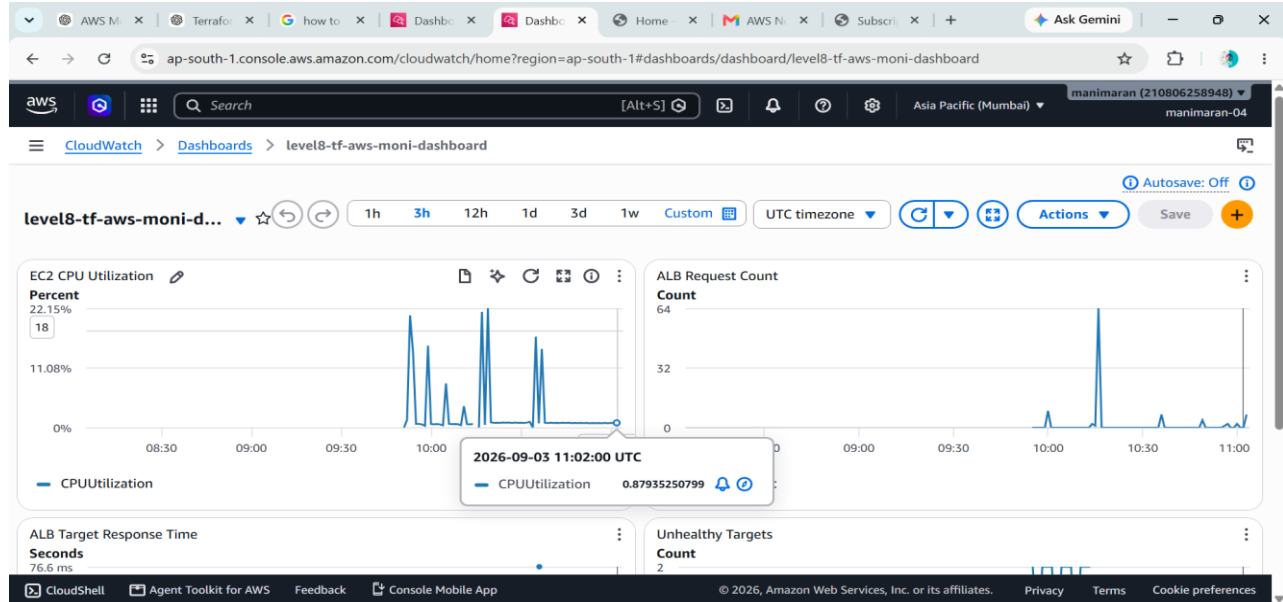


Figure 61. CloudWatch dashboard with CPU, ALB request count, response time and unhealthy-target widgets.

Step 61 – CloudWatch dashboard verification

Instruction / Action: Continue dashboard verification and confirm that the displayed time-series metrics contain operational data from the deployed environment.

Expected Result: The monitoring dashboard provides usable visibility into application and load-balancer behavior.

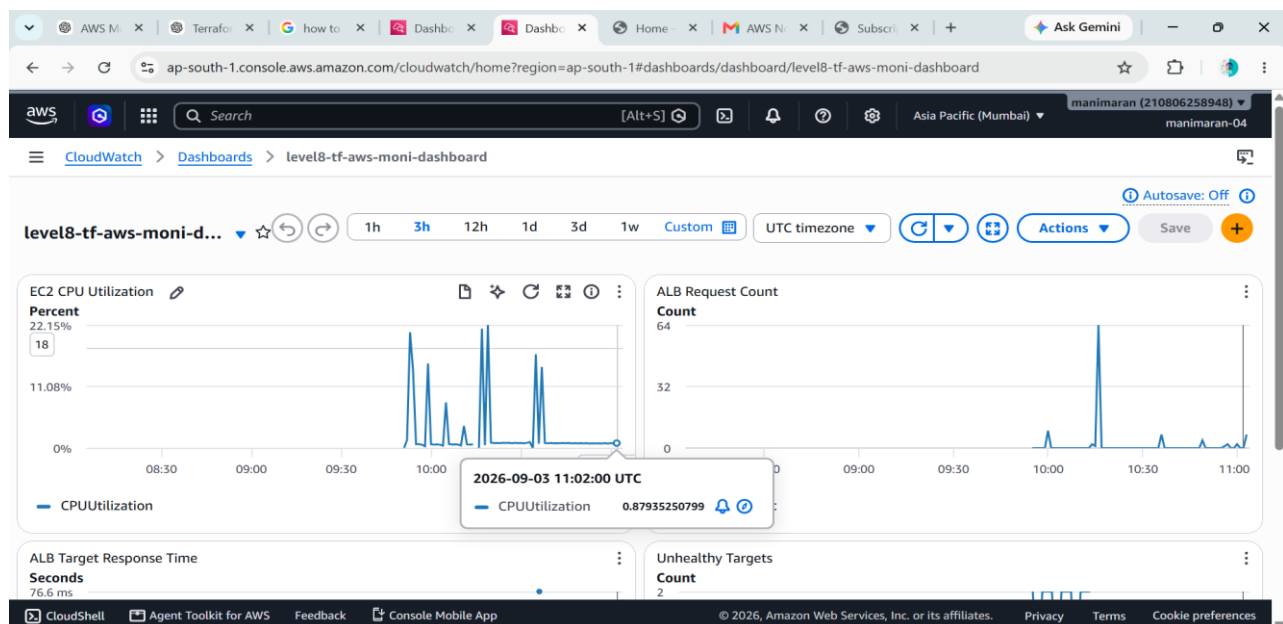


Figure 62. CloudWatch dashboard verification.

Step 62 – Terraform outputs showing ALB, ECR, SNS, subnet, target-group and VPC identifiers

Instruction / Action: Run Terraform outputs and verify that important identifiers such as the ALB DNS name, ECR details, SNS ARN, subnet IDs, target-group ARN and VPC ID are exposed.

Expected Result: Terraform outputs provide the resource identifiers needed for validation and operations.

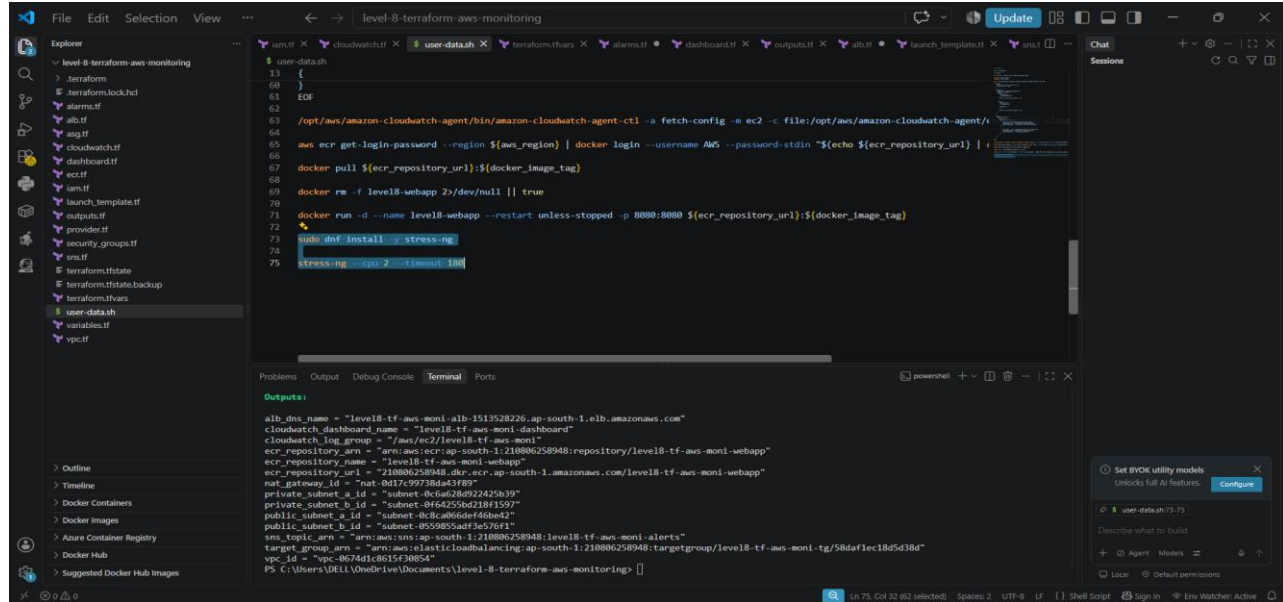


Figure 63. Terraform outputs showing ALB, ECR, SNS, subnet, target-group and VPC identifiers.

Step 63 – SNS/CloudWatch alarm notification email evidence

Instruction / Action: Open the email inbox and verify that the SNS/CloudWatch alarm notification was received.

Expected Result: An alarm state-change notification is delivered through SNS to the configured email subscription.

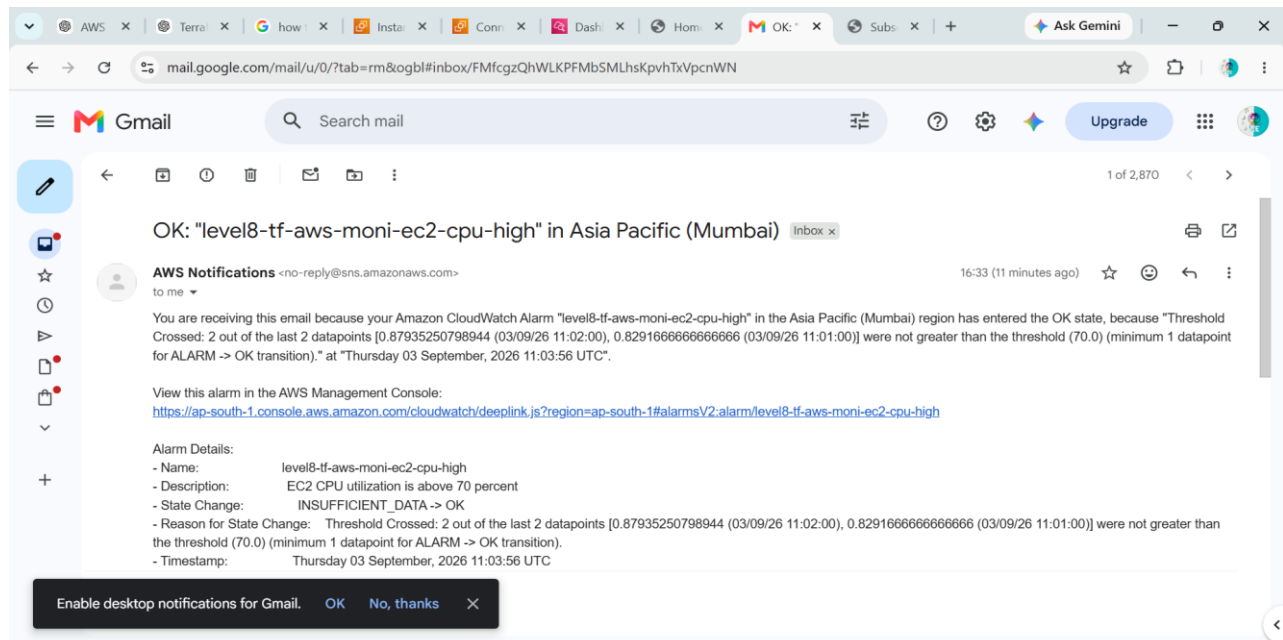


Figure 64. SNS/CloudWatch alarm notification email evidence.

Step 64 – CloudWatch alarm email details and metric configuration

Instruction / Action: Open the alarm notification email and inspect the alarm name, state-change information and metric details included in the message.

Expected Result: The notification contains enough information to identify the CloudWatch alarm event and its monitored condition.

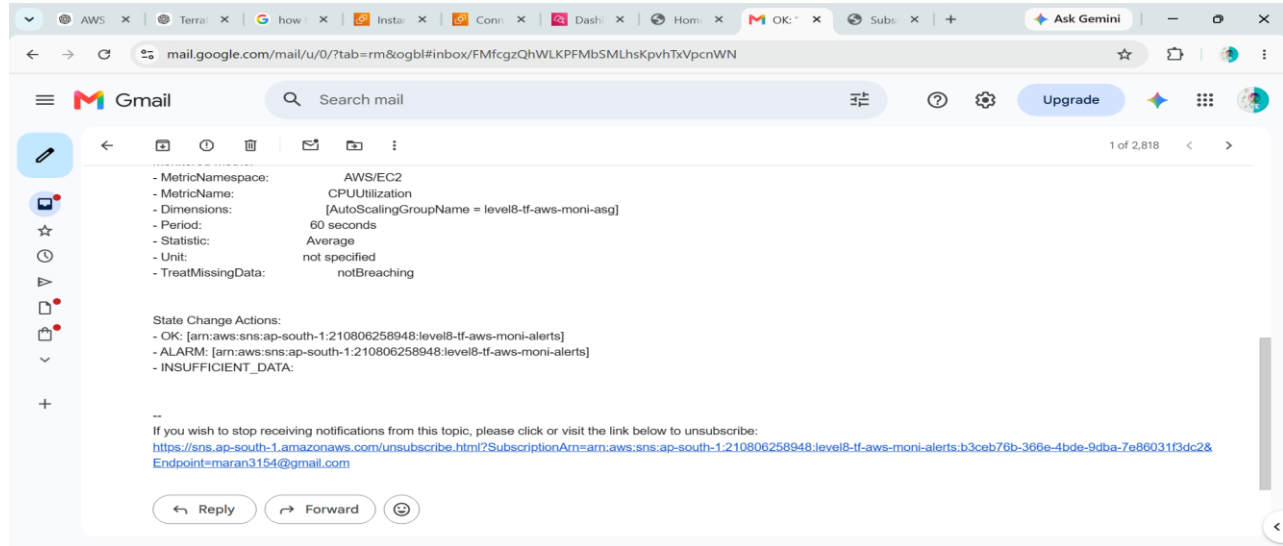


Figure 65. CloudWatch alarm email details and metric configuration.

Step 65 – Terraform destroy progress and final resource cleanup

Instruction / Action: After completing validation, run terraform destroy to remove the test environment. Monitor the Terraform output until the dependent AWS resources are destroyed.

Expected Result: Terraform completes the cleanup and reports the configured resources as destroyed; the supplied evidence records 39 resources destroyed.

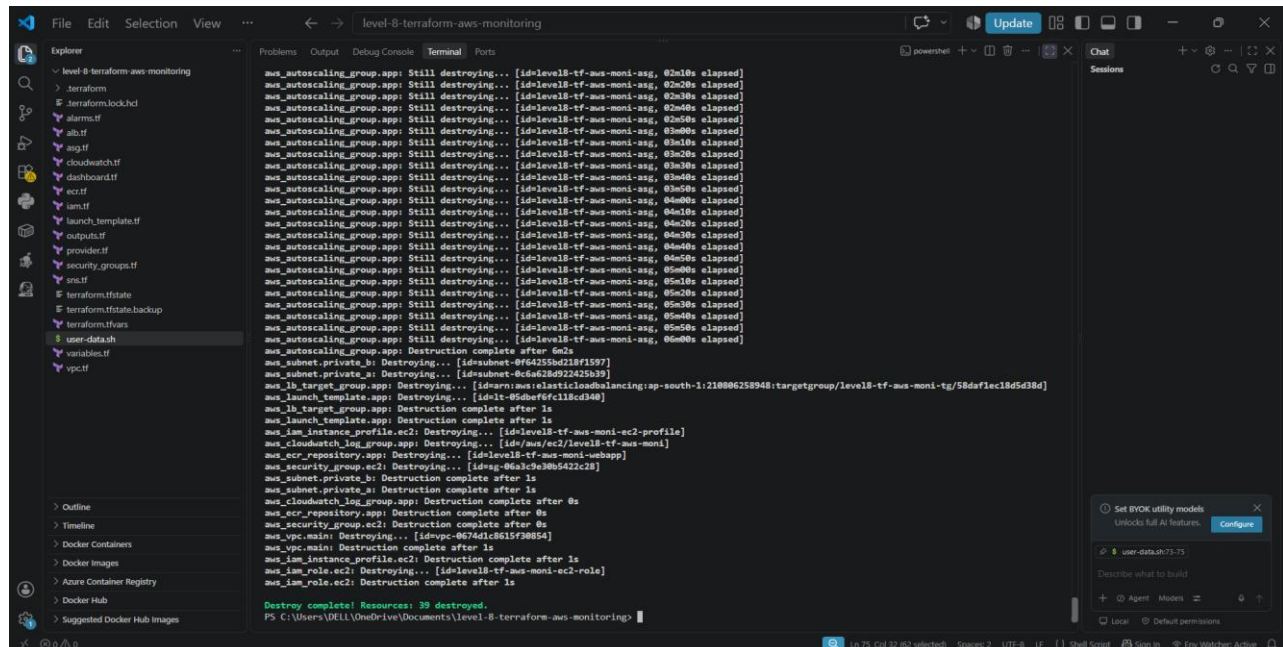


Figure 66. Terraform destroy progress and final resource cleanup.

Step 66 – EC2 console verification after destruction; no matching running instances

Instruction / Action: Open the EC2 console after destruction and verify that the project instances are no longer running/present.

Expected Result: The final AWS console check shows no matching running application instances, confirming the environment cleanup.

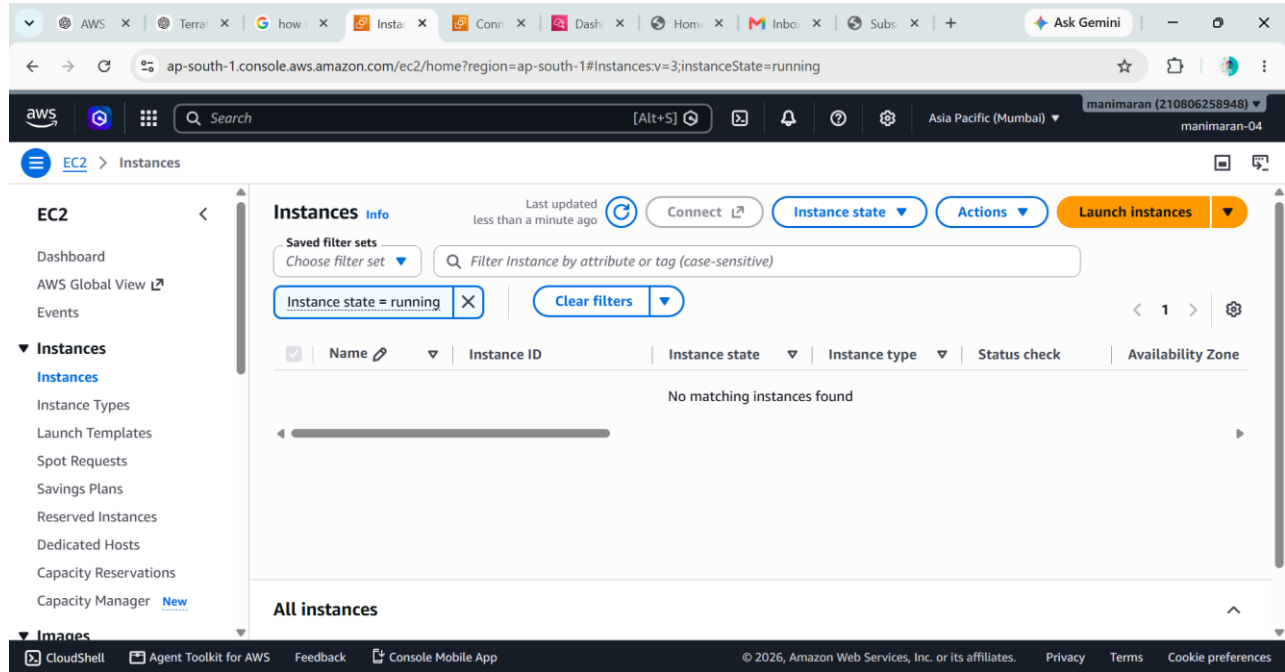


Figure 67. EC2 console verification after destruction; no matching running instances.

12. Monitoring and Alerting Design

The monitoring path keeps the private application tier observable while the public ALB remains the client entry point. The CloudWatch Agent adds system metrics and log collection, CloudWatch alarms evaluate operational conditions, and SNS delivers state-change notifications.

- EC2 CPU utilization alarm: threshold 70%, 2 evaluation periods and 2 datapoints to alarm.
- EC2 status-check alarm: monitors failed EC2 status checks.
- ALB 5XX alarm: monitors server-side HTTP errors reported by the load balancer.
- Target-unhealthy alarm: monitors unhealthy registered application targets.
- ALB response-time alarm: monitors application latency.
- SNS topic: level8-tf-aws-moni-alerts.
- Email subscription: confirmed successfully in the supplied evidence.
- CloudWatch dashboard: consolidated operational view.

13. Application Request and Monitoring Flows

Application Request Flow: Browser / Client → ALB :80 → Target Group → Private EC2 Instance → Docker Container → Tomcat :8080 → Java Web Application

Image Deployment Flow: Docker Build/Tag → Amazon ECR → EC2 IAM Instance Role → ECR Login → Docker Pull → Docker Container → Tomcat :8080

Monitoring Flow: EC2 + ALB → CloudWatch Metrics/Logs → CloudWatch Alarms → SNS Topic → Email Subscription

Infrastructure Provisioning Flow: Terraform Configuration → terraform apply → VPC/Subnets/Routes → Security Groups → ECR/IAM → ALB/Target Group → Launch Template → ASG → CloudWatch/SNS/Dashboard

14. Verification Checklist

- ✓ Terraform configuration files created and organized by AWS component.
- ✓ AWS region configured as ap-south-1.
- ✓ VPC and public/private subnet configuration defined.
- ✓ Internet Gateway and NAT Gateway configured.
- ✓ ALB and target group configured.
- ✓ EC2 launch template configured with IAM instance profile.
- ✓ Auto Scaling Group configured with desired capacity 2 and maximum capacity 4.
- ✓ ECR repository created.
- ✓ Docker image successfully pushed to ECR after correcting the initial repository-name mismatch.
- ✓ SNS topic created and email subscription confirmed.
- ✓ CloudWatch Agent configuration included in user data.
- ✓ Four CloudWatch alarms visible and in OK state in the supplied evidence.
- ✓ CloudWatch dashboard created and populated with operational metrics.
- ✓ Alarm email evidence received successfully.
- ✓ Terraform outputs produced resource identifiers.
- ✓ Terraform destroy completed with 39 resources destroyed.

15. Troubleshooting / Important Observations

- The ECR push evidence shows an initial repository-name mismatch; the image was then retagged with the correct ECR repository URI and successfully pushed.
- The CPU alarm email shows an OK transition when the observed CPU datapoints were below the 70% threshold.
- The CloudWatch dashboard shows CPU utilization, ALB request count, ALB target response time and unhealthy-target metrics.
- The final Terraform destroy took several minutes while the Auto Scaling Group was being removed, after which dependent resources were destroyed successfully.

16. Security Notes

Some supplied screenshots may display account identifiers, email addresses, repository details, subscription information or other environment-specific values. Before sharing this report publicly, review the embedded screenshots and redact credentials or sensitive identifiers that should not be distributed. Never commit AWS access keys, tokens or private keys to GitHub.

17. Key Lessons Learned

- Terraform makes AWS infrastructure repeatable, reviewable and easier to destroy cleanly.
- Separating public ALB resources from private application EC2 resources improves the network architecture.
- Security-group references can restrict application traffic so Tomcat is reachable from the ALB.
- ECR provides a private AWS-native container registry for the application image.
- IAM instance profiles avoid hard-coding long-lived AWS credentials on EC2.
- User data can automate Docker installation, CloudWatch Agent configuration, ECR authentication and application startup.
- CloudWatch dashboards provide a single operational view of infrastructure and load-balancer behavior.
- CloudWatch alarms plus SNS provide automated operational notifications.
- Terraform outputs make important deployed resource identifiers easy to retrieve.
- Destroying the stack after testing helps control AWS costs and confirms that resource dependencies are modeled correctly.

18. Interview / Viva Explanation

I built an AWS monitoring-focused infrastructure project using Terraform. Terraform provisions a VPC with public and private subnets, an Application Load Balancer, target group, Auto Scaling Group, EC2 launch template, ECR repository, IAM role, CloudWatch monitoring, alarms, dashboard and SNS notifications. The application is packaged as a Docker image and stored in ECR. Private EC2 instances authenticate to ECR using an IAM instance profile, pull the image and run Tomcat on port 8080. The ALB receives HTTP traffic on port 80 and forwards it to the private instances. CloudWatch collects metrics and logs, while alarms send state-change notifications through SNS to email. Terraform destroy was then used to remove the complete test environment.

19. Conclusion

The Level 8 project demonstrates a complete Terraform-managed AWS application and monitoring stack. The Screenshots covers infrastructure definition, VPC networking, security groups, ECR image storage, IAM permissions, EC2 bootstrap, ALB and Auto Scaling, CloudWatch Agent configuration, CloudWatch alarms, dashboard creation, SNS notification delivery, Terraform outputs and final infrastructure destruction. The project therefore provides practical experience with Infrastructure as Code and production-oriented AWS monitoring concepts.

Appendix – Complete Infrastructure Flow

